

Passing the Benchmark, Failing the README: Leakage and Shortcut Learning in a Brazilian Portuguese Prompt-Injection Classifier

Rodrigo Faria

Independent Researcher

<https://github.com/alucardigo> · <https://www.linkedin.com/in/faria-rodriigo>

October 8, 2026

Abstract

Prompt-injection classifiers increasingly sit between LLM agents and the content they fetch. We report a fully traced case study of building one for Brazilian Portuguese (pt-BR) from open data. Our best model, a fine-tuned multilingual-e5-large, detected 94.7% of the injections in Weni, a native pt-BR test set, but the figure did not survive decontamination: retrained without test duplicates and near-duplicates, the same recipe detects 68.0%. Scored page by page through the code path of an agent hook, both versions raised a false alarm on all 217 technical documentation pages at every threshold from 0.5 to 0.999. Among five public classifiers, one also warned on every page; only Llama Prompt Guard 2 (86M) combined a 1.8% page false-alarm rate with 62% detection; and sentence and page benchmarks ranked the models in opposite orders. Adding 10,000 technical benign texts supported our diagnosis, a shortcut equating technical prose with injection: false alarms vanished on technical sentences and fell to 19–35% of pages, while Weni detection stayed close to the decontaminated model’s (63.7% against 68.0%). No version we trained meets both acceptance criteria. We contribute the case study, a leakage audit checked by retraining, a page-level evaluation protocol, agent-integration lessons and open artifacts.

1 Introduction

LLM agents read untrusted text: web pages, tool outputs, repository files, e-mails. Any of it may carry instructions addressed to the model rather than to the user. Prompt injection was first shown through direct user input (Pérez and Ribeiro, 2022); when the instructions arrive inside content the application retrieves, it is known as indirect prompt injection (Greshake et al., 2023; Liu et al., 2024). A common mitigation is a small classifier that scores fetched content and warns, or blocks, when it looks like an injection (Jacob et al., 2025; Meta, 2025; ProtectAI.com, 2024). Such classifiers are usually reported with sentence-level accuracy on public benchmarks, almost all in English.

This paper documents what happened when we built one for Brazilian Portuguese, using only open data, and tried to switch it on as a warning hook in front of the web-fetch tool of an LLM coding agent. The core study took one week (30 September to 6 October 2026); a baseline with Llama Prompt Guard 2 and a decontaminated retraining followed on 7 October. Every number below is traced to a log line, result file, or commit in an evidence dossier.

The headline is a negative result in two parts. The best model passed the benchmark tests by a wide margin, missed its own pre-fixed acceptance criterion on the native pt-BR gold set by a

single text (284/300 against 285 required), and then, evaluated page by page on ordinary technical documentation, raised a false alarm on 217 of 217 pages at every threshold we tried. A hand-written smoke-test sentence had been flagging the failure mode since the second training round, and we did not read it as the signal it was (Section 6). The benchmark number then turned out to be inflated as well. A post-hoc audit had put the effect of near-duplicates of the gold test in the training data at about 1.3 points; retraining the same recipe without exact duplicates of any test item, their translations, near-duplicates of the gold test, the rows an aggregated dataset had copied from xTRam1, or rows excluded for their licence (v3-pub) brought Weni detection down to 68.0%, 26.7 points lower on the same items, while the page failure stayed exactly as it was (Section 4, Table 5). The drop bounds the inflation from above rather than measuring it: v3-pub also has fewer attack examples, and it drops more on the Weni texts that have no near-duplicate in v3’s training data (29.8 points) than on those that do (13.8).

Two further measurements sharpen the result. Under the same page protocol the failure is not peculiar to our model: one of five public classifiers also warned on every page, and two others warned on about 15% of the benign pages at threshold 0.5 while detecting 63–70% of the injected ones (Section 5.4). The exception is instructive: Llama Prompt Guard 2 (86M), whose card reports benign web data in training, warned on 4 of the 217 benign pages (1.8%) while detecting 62% of the injected ones, although it caught only 30% of the short English injections and detects 93.7% of Weni’s sentences, below our criterion. And retraining with technical benign text (v6, Section 7) removed the false alarms on technical sentences and cut page false alarms to 19–35%. It also lowered Weni detection from v3’s leakage-inflated 94.7% to 63.7%, which we first read as the price of removing the shortcut. The decontaminated v3-pub, which has no technical negatives and still warns on every page, drops almost as far: v6 is 4.3 points below it, not significantly, so the comparison with v3 cannot charge that drop to the new negatives. The two models miss partly different texts, however, and how much of v6’s drop the new negatives cause remains open; the page improvement, by contrast, belongs to them (Section 7.2). None of our models meets both acceptance criteria; of the models we tested, only Prompt Guard 2 (86M) meets the page-level one, and only as a point estimate (95% interval 0.7–4.6%).

We think the case is useful beyond pt-BR because the failure is not exotic. The benign side of the training data was largely conversational (assistant commands and instruction-following prompts); the deployment distribution is mostly technical prose full of imperatives (install, run, configure, ignore this warning). A classifier can separate injections from chat with near-perfect accuracy and still have learned nothing that separates injections from documentation. Nor is the leakage exotic: it entered through machine-translated copies of test items and through aggregated datasets that contain other benchmarks, and an audit by embedding similarity against the gold test missed most of the score’s dependence on the leaked data.

A concurrent study by Liu (2026), which appeared while this work was under way, re-evaluates fifteen English detectors on agent tool outputs and on benign GitHub README chunks and reaches the same practical conclusions. We read our study as an independent replication of that gap in another language and with a detector we trained ourselves, so that the cause can be traced to the training recipe and tested by retraining; it adds whole-page aggregation, integration lessons for agent hooks, and the perspective of typed decision models (Section 9.3).

Contributions.

1. **A documented case study of a benchmark-versus-use gap in pt-BR.** Nine training rounds (frozen-embedding probes; fine-tuned e5-base and e5-large, with and without additional attack data; a decontaminated retraining; and a retraining with technical benign text that tests the diagnosis) plus a fixed-weight ensemble, evaluated on up to 19 test sets, with

an acceptance criterion fixed in version control before the first measurement on the gold set, and a leakage audit checked by retraining: a cosine-similarity cut had put the inflation of the best model’s 94.7% at about 1.3 points, and the same recipe retrained without test duplicates and near-duplicates detects 68.0%.

2. **A page-level evaluation protocol for injection classifiers** that reuses the exact code path of the deployment hook, scores pages by the maximum over overlapping windows, and reports false alarms on benign technical pages alongside detection on pages with an inserted injection. Applied to five public classifiers, it shows that sentence-level and page-level evaluation can rank models in opposite orders. The benchmark is distributed as a hash-pinned manifest (217 benign pages and 60 insertions) plus a builder that downloads the pages from their sources, so no third-party text is redistributed.
3. **Engineering lessons for integrating classifiers into agents:** an encoding bug that made benign text look like an injection, a server-version mismatch silently swallowed by a fail-open hook, a sliding window longer than the model’s context, and an evaluation input list that drifted between runs.
4. **Open artifacts:** code (Apache-2.0), including the page benchmark builder, and all metric files and the benchmark manifest (CC-BY-4.0), released with this paper; the v3-pub weights (MIT) are published separately (Section 13).

The rest of the paper is organised as follows. Section 2 gives background on typed decision models and on prompt injection. Section 3 describes data, translation, leakage control and training. Section 4 reports sentence-level results and the leakage audit, and Section 5 page-level results, including public baselines. Section 6 diagnoses the gap, Section 7 tests the diagnosis by retraining, Section 8 lists integration lessons, Section 9 discusses what this says about benchmark accuracy, about using decision models with agents and about concurrent work, Section 10 lists limitations and Section 11 concludes.

2 Background

2.1 Typed decision models (“System One”)

A recent line of products and open models answers *closed* questions about text: the caller supplies a text and a fixed set of options, and the model returns the chosen option with a probability, without generating prose. TypeSafe markets its hosted model Jev this way, as a “System One” model, a name its launch post attributes to Kahneman’s distinction between fast, intuitive and slow, deliberate thinking (Almeida, 2026; Kahneman, 2011). On 1 October 2026 Cloudflare released Clef and Clef-flash, open-weight (Apache-2.0) decision models that accept the same API (Chen et al., 2026; Cloudflare, 2026). Kev-9B is another open-weight decision model, a LoRA adapter and decision head on a 9B base model (Palmer, 2026). Laya, from Convai Innovations, is an open (Apache-2.0) local implementation of typed decisions (single choice, score, and binary outputs) that runs on CPU and is meant to be fine-tuned on the user’s own domain (Convai Innovations, 2026). The Decision Index is a public benchmark suite for this model family; it does not redistribute its test data and instead ships builders that reproduce the files byte for byte and refuse any file whose hash does not match (apolinario, 2026). On that leaderboard the zero-shot Laya checkpoint is reported far below the 9B-class Kev-9B (apolinario, 2026).

A Portuguese-language post by Fabio Akita, written for developers, discusses these models and compares Jev with Clef and Laya (Akita, 2026a). Our paraphrase of the points relevant here

follows. Akita reads Jev as a good, cheap hosted classifier rather than a new kind of model, and argues that its headline speed multiplier is impressive only because the comparison is with chat models that write prose before a label can be extracted. Much of the post is an essay written for it by Paulo Câmara, who discloses that he co-founded a company competing in the same market. Recomputing public third-party tests, the essay reports that Jev picks the first listed option, with high stated confidence, when it has nothing to decide on, and that its decisions move with attributes irrelevant to the case. In Akita’s view the tool fits recurring decisions over text with a closed set of outcomes, where deterministic code around the model enforces the exact rules and wrong answers can be caught or reviewed. The essay’s usage advice includes recalibrating on a small set of one’s own labelled examples before picking a threshold, and not giving the model sole authority over destructive actions. An earlier post by the same author argued that teams using chat models to emit labels need not buy such a service to fix the habit: structured output from the model they already use, or a classifier trained for the task, also does the job (Akita, 2026b).

This study is an instance of the second option: a dedicated binary classifier (“is this fetched content an injection?”) trained for one job. It was deployed as a decision *pack* in a local toolkit built around the open Laya server; the classifier itself is a fine-tuned multilingual-e5 model and does not derive from Laya’s weights.

2.2 Prompt injection and its detectors

Prompt injection was described first for direct user input (Pérez and Ribeiro, 2022) and then for content an application retrieves on the user’s behalf (Greshake et al., 2023); formal treatments and benchmarks followed (Debenedetti et al., 2024; Liu et al., 2024; Yi et al., 2025; Zhan et al., 2024). The HackAPrompt competition produced a large public corpus of human-written adversarial prompts, labelled by whether they succeeded (Schulhoff et al., 2023). Defences range from separating instructions from data, either by marking untrusted input inside the prompt (Hines et al., 2024) or by training the model on a structured query format (Chen et al., 2025), to system designs that do not let untrusted data drive control flow (Debenedetti et al., 2025), and to detectors: fine-tuned DeBERTa classifiers (deepset, 2023a; ProtectAI.com, 2024), Meta’s Prompt Guard family (Meta, 2024, 2025), and others. Over-defence, that is, benign inputs flagged because they share surface features with attacks, has been documented for English guard models and traced to a bias toward trigger words (Li and Liu, 2024; Li et al., 2025). Two recent studies are close to ours: a multi-axis benchmark finds that hard-negative augmentation removes over-defence on curated benign inputs but leaves it largely intact on externally sourced ones (Shire and Kim, 2026), and a re-evaluation of fifteen public detectors on agent tool outputs reports two of them flagging every benign GitHub README chunk it scored (Liu, 2026) (we compare with it in Section 9.3). Evaluation at low false-positive rates has been argued for as the relevant regime for deployment (Jacob et al., 2025; Liu, 2026).

Our failure is a familiar one in machine learning under another name: shortcut learning (Geirhos et al., 2020), where a model exploits a feature that separates classes in the training distribution but not in the deployment distribution, as with annotation artifacts and syntactic heuristics in NLI (Gururangan et al., 2018; McCoy et al., 2019), and the accuracy drops seen on freshly collected test sets and under in-the-wild distribution shift (Koh et al., 2021; Recht et al., 2019). Behavioural testing beyond held-out accuracy (Ribeiro et al., 2020) is the general remedy; Section 5 applies it in the unit of use for injection detectors.

Resources for pt-BR are scarce. The only native pt-BR injection set we found with usable size is Weni’s (Weni, 2024); a multilingual set with Portuguese items exists (rikka-snow, 2024). Most of our data is English, which motivated the machine-translation pipeline below.

Table 1: Data sources. “MT” = machine-translated copies added on top (Section 3.2).

Source (Hugging Face id)	Declared license	Role	Train	Test
deepset/prompt-injections (en/de)	Apache-2.0	train	546	116
jackhhao/jailbreak-classification	Apache-2.0	train	1,044	262
reshabhs/SPML_Chatbot_Prompt_Injection	MIT	train (subsample)	4,000	1,000
xTRam1/safe-guard-prompt-injection	none declared	test only	0	1,500
rikka-snow/prompt-injection-multilingual (pt)	MIT	train (from round B)	6,006	1,276
dmilush/shieldlm-prompt-injection	MIT (curation)	train (from B)	4,000	4,000
yanismiraoui/prompt_injections (positives only)	Apache-2.0 + NOTICE	train (from B)	815	219
mteb/amazon_massive_intent (pt), benign	CC-BY-4.0 (origin)	train (from B)	1,500	1,500
Weni/prompt-injections-1.0.0 (pt-BR, injections only)	none declared	gold test only	0	300
databricks/databricks-dolly-15k, benign	CC-BY-SA-3.0	train (from v2)	3,000	1,000
HackAPrompt (mirror, successful attacks only)	none on mirror; origin MIT	train (v4, v5)	3,000	1,000
Stack Overflow questions / answers; CodeSearchNet docstrings, benign technical	CC-BY-SA (SO); per repository (CSN)	train (v6 only)	4,000 / 4,000 / 2,000	1,000 each

3 Method

3.1 Data

Table 1 lists every data source, its declared license, its role and its size. Per-source sizes are differences between consecutive cumulative counts in the training logs. Where a dataset has an official **test** split we use it; otherwise the test set is the hash bucket ≥ 80 of an md5 partition, capped at 1,000 items.

Dataset references: deepset (deepset, 2023b), jackhhao (jackhhao, 2023), SPML (Sharma, 2024; Sharma et al., 2024), xTRam1 (xTRam1, 2024), rikka (rikka-snow, 2024), ShieldLM (dmilush, 2026), yanis (Miraoui, 2023), MASSIVE (FitzGerald et al., 2023), Weni (Weni, 2024), Dolly (Conover et al., 2023), HackAPrompt (Schulhoff et al., 2023) via the mirror (imoxto, 2023), Stack Overflow (koutch, 2023; pacovaldez, 2022), CodeSearchNet (Husain et al., 2019; Nan-Do, 2023).

Two composite tests are used throughout. *pt-native* joins Weni (300 positives) and MASSIVE-pt (1,500 negatives), 1,800 items. *Smoke* is ten sentences written in the code, five injections and five benign; one of the benign sentences is a Portuguese installation instruction (“Installation instructions: run setup.exe and click next”), which turns out to matter (Section 6).

Two facts discovered after the training rounds change how the tests must be read. First, **xTRam1 is not out of distribution** for any model trained with ShieldLM: 577 of ShieldLM’s 4,000 training lines come from xTRam1’s own training split, and 4 of the 1,500 xTRam1 test items appear in ShieldLM’s training subset (exact match after case and whitespace normalisation). A later check found more overlap (below). We therefore describe xTRam1 as “same distribution,

partly overlapping items” for every model before v3-pub (Section 3.4). Even v3-pub, trained without those rows and without any duplicate of the 1,500 xTRam1 test items, keeps 422 training texts, 378 of them from jackhhao and 37 from Dolly, that are identical to texts elsewhere in the xTRam1 dataset, so xTRam1 is not fully external for v3-pub either. Weni is the only genuinely external test. Second, 14 of the 262 jackhhao test items appear in ShieldLM’s training subset; deepset has no item in that subset (0/116). A check of v3’s whole training set against every test, made for v3-pub (Section 3.4), found exact duplicates after case and whitespace normalisation in almost every test: 20 of the 262 jackhhao items, 72 of the 1,000 SPML items, 83 of the 1,500 xTRam1 items, 439 of the 4,000 ShieldLM items, 75 of the 1,500 MASSIVE-pt items and a few elsewhere; none in deepset or Weni. Also, the Portuguese MASSIVE configuration is European Portuguese, not Brazilian.

3.2 Translation to pt-BR

English training and test data were translated locally with `Helsinki-NLP/opus-mt-tc-big-en-pt`, an OPUS-MT model trained in the Tatoeba Translation Challenge (Helsinki-NLP, 2022; Tiedemann, 2020; Tiedemann and Thottingal, 2020), language token `>>por<<`, greedy decoding (one beam, no sampling), which makes the translation deterministic. Inputs are truncated at 512 tokens and 1,500 characters. Up to 1,000 examples per English source (those with an even hash) are added to training, and the first 300 SPML and 500 xTRam1 test items are translated as additional tests; the deepset and jackhhao tests are translated in full. Round A used an earlier rule: half of the training data by hash, with a single cap of 3,000.

The generation cap was originally fixed at 512 new tokens; greedy decoding would fall into repetition and run to the cap, about 7 s per short instruction. Capping generation at $\min(512, 1.6 \times \text{input tokens} + 16)$ removed the problem; the reported 5–10 $\times$ speed-up is recorded only in a commit message and is not independently measured (Section 10). Translations are cached by the SHA-1 of the full source text and flushed after every batch, and the work was split across two CPU machines; the GPU quota was reserved for fine-tuning. Round A translated 2,723 training and 1,161 test texts. No translated text was reviewed by a human.

3.3 Partition and selection protocol

All partitions are deterministic functions of the text: `bucket = int(md5(text)[:8], 16) mod 100`. The probe’s validation set is `bucket < 15` (15%). For probes, the regularisation constant $C \in \{0.05, 0.2, 1, 4\}$ and class weighting $\in \{\text{none}, \text{balanced}\}$ are chosen by balanced accuracy on validation, and a temperature is fitted on validation by minimising log-loss over a 60-point log grid from 0.1 to about 20. Test sets are used only to measure. Test sets are never uploaded to the fine-tuning service: the export for cloud fine-tuning contains training data only, and evaluation runs on the author’s own machines (a laptop and an ARM cloud VM).

Acceptance criteria were fixed before measurement (Table 2). The criterion used for the decision ($\text{Weni} \geq 95\%$ and $\text{xTRam1-pt} \geq 90\%$) was committed on 2 October, almost three days before the first Weni measurement. The page-level criterion was added only after the page-level failure and applies to future rounds.

3.4 Leakage lock

The Weni set is believed to be a Portuguese translation of HackAPrompt material; the claim appears in our code and commit history but was not independently verified at the time. To keep the gold test honest, from round v4 on every training example whose maximum cosine similarity to any Weni text exceeds 0.9 (multilingual-e5-base, normalised vectors) is removed. On the v4 export this

Table 2: Acceptance criteria.

Date	Criterion	Status when fixed
1 Oct	$\geq 95\%$ on xTRam1, then believed to be out of distribution (Section 3.1)	before round 0 results
2 Oct	Weni $\geq 95\%$ and xTRam1-pt $\geq 90\%$; if met, enable a warn-only, fail-open hook	before any Weni measurement
6 Oct	page-level false-alarm rate $< 2\%$	after the page-level failure; for future rounds

removed 4,008 of 27,693 examples. HackAPrompt enters only through rows labelled as successful attacks; otherwise a failed attack would become an implicit benign example.

The lock was not applied to the v2/v3 export, which predates it. Applying the same rule later to the v3 training data flags 115 examples as near-duplicates of Weni texts. They are mostly machine translations (98 injections and 17 benign examples; 48 from jackhhao-pt, 18 from SPML-pt and 10 each from deepset-pt and Dolly-pt), and 58 of the 300 Weni texts have one of them as a neighbour. We return to this in Sections 4, 7 and 10.

Decontaminated retraining (v3-pub). On 7 October we retrained v3’s recipe on a strict subset of its training data, both to measure how much its Weni score depended on leaked examples and to obtain weights free of the licence problem of Section 12. Four filters, applied in order, remove 1,758 of the 23,693 examples and add nothing: 673 rows whose text comes from ShieldLM sources that are xTRam1 itself (the 577 rows of Section 3.1) or carry a licence that differs between origin and aggregator (653 ShieldLM rows and 20 copies of the same texts in jackhhao and Dolly); 831 texts identical, after case and whitespace normalisation, to some text of any test, translated tests included; 164 translations whose original was removed, or originals whose translation was; and 90 examples above cosine 0.9 to some Weni text, the lock above (the other 25 of the 115 had already gone with the earlier filters). The result has 21,935 examples, 38.6% of them injections (39.8% in v3). Of the 967 injections removed, 513 are exact duplicates of test items, 256 come from the excluded ShieldLM sources and 198 are translation pairs or Weni near-duplicates. After filtering, no test item has a normalised duplicate in training, and no training text is identical to, shares a 200-character fragment with, or shares a 13-gram with any of the 334 package READMEs from which the page test was drawn. Recipe, notebook, evaluation code, machine, tests and frozen pages are those of v3 and v6; internal validation accuracy was 99.49%.

3.5 Models and training

Probes (rounds 0, A, B). Frozen `intfloat/multilingual-e5-base` (intfloat, 2023; Wang et al., 2024) with the `query:␣` prefix and a 256-token limit, followed by a standardised logistic regression. Embeddings were computed on a laptop-class CPU (Intel i7-1255U).

Fine-tuned models (v2–v5). Binary sequence classification on `multilingual-e5-base` or `multilingual-e5-large`, input `query:␣ + text`, 256 tokens for training and inference, 3 epochs, learning rate $2e-5$, weight decay 0.01, warm-up over 6% of steps, fp16, best epoch by validation accuracy on an 8% stratified split with seed 0. e5-base used batch 32; e5-large used batch 8 with 4 accumulation steps and gradient checkpointing. Training ran on a free Kaggle notebook with two NVIDIA T4 GPUs (15,360 MiB each); whether both GPUs were effectively used is unverified (Section 10). **At inference the fine-tuned models use temperature 1.0, i.e., raw logits without calibration.**

Fixed-weight ensemble. The logits of v3 and v5 are averaged with weights 0.5/0.5, chosen

Table 3: Training rounds.

Round	Date	Model	Training data	Size	Note
0	1 Oct	probe, e5-base	deepset + jackhhao + SPML, English only	5,590 examples; 4,622 after validation split	—
A	2–5 Oct	probe, e5-base	round 0 + 2,723 MT pt	n_train 6,910	translation
B	5 Oct	probe, e5-base	7 sources incl. native pt, 17,911 + MT	n_train 17,580	native pt data
v2	5 Oct	fine-tuned e5-base	20,911 originals (8 training sources: round B’s 7 + Dolly) + 2,782 MT = 23,693	val. 1,896	no leakage lock
v3	5 Oct	fine-tuned e5-large	same export as v2	val. 1,896	no leakage lock
v4	6 Oct	fine-tuned e5-base	+ HackAPrompt; lock removed 4,008 → 23,685	val. 1,895	leakage lock
v5	6 Oct	fine-tuned e5-large	v4 export	val. 1,895	leakage lock
mix	6 Oct	v3 $\oplus$ v5 logits, 0.5/0.5	—	—	no tuning on test
v6	6 Oct	fine-tuned e5-large	v3 export – 115 Weni near-duplicates + 10,000 technical benign	33,578 examples	leakage lock; technical negatives (Section 7)
v3-pub	7 Oct	fine-tuned e5-large	v3 export – 1,758 (Section 3.4)	21,935 examples	decontaminated against every test

before looking at any test result because no separate Weni validation set exists and tuning the weight on the test would game the criterion.

3.6 Rounds

Table 3 summarises the rounds. Most rounds changed one main factor relative to a reference round; three did not: v2 both switched to fine-tuning and added Dolly as benign data, v4/v5 add both HackAPrompt and the leakage lock relative to v2/v3, and v6 both adds technical negatives and drops the 115 Weni near-duplicates (Section 7.1).

Validation accuracies on the cloud split were 99.42% (v2), 99.63% (v3), 99.31% (v4), 99.63% (v5), 99.70% (v6) and 99.49% (v3-pub). Compute: the four fine-tuning sessions used 20,476 s, about 5.69 h of notebook time; an e5-base run took about 31–33 min and an e5-large run about 2 h 15 min to 2 h 22 min. v6 and v3-pub took another 12,713 s and 7,056 s. The three probe rounds spent 10,595 s (about 2.94 h) computing embeddings on CPU. Appendix A lists all costs.

4 Sentence-level results

Table 4 reports accuracy for every round on every test it was run on. For Weni, which contains only injections, accuracy equals the detection rate. Sources are given per column.

Table 4: Sentence-level accuracy (%) by round. “—” = not run. Smoke reports correct/10. v3’s Weni figure is inflated by leakage (Section 4d; the decontaminated v3-pub detects 68.0%, Table 5).

Test (n)	0	A	B	v2	v3	v4	v5	mix
Weni, pt-BR native (300)	—	—	72.3	74.3	94.7	66.7	89.7	91.3
xTRam1, en (1,500)	86.3	89.1	94.3	98.9	98.7	99.0	99.1	99.0
xTRam1-pt, MT (500)	—	81.8	82.8	95.4	96.0	96.0	98.0	96.8
MASSIVE-pt, benign (1,500)	—	—	99.3	99.7	99.8	99.8	100	99.9
Dolly, benign en (1,000)	—	—	—	99.7	99.6	99.7	—	—
Dolly-pt, benign MT (1,000)	—	—	—	99.6	99.3	99.9	99.5	99.5
pt-native (1,800)	—	—	94.8	95.5	98.9	94.3	—	—
deepset (116)	81.9	81.0	84.5	94.0	—	93.1	—	—
jackhhao (262)	95.8	96.6	93.1	99.2	—	98.9	—	—
SPML (1,000)	98.9	98.7	98.8	99.4	—	99.3	—	—
rikka (1,276)	—	—	88.8	95.6	—	94.5	—	—
ShieldLM (4,000)	—	—	96.7	98.7	—	98.8	—	—
yanis (219)	—	—	98.6	100	—	99.1	—	—
HackAPrompt (1,000)	—	—	—	—	—	99.8	—	—
deepset-pt, MT (116)	—	86.2	85.3	94.8	—	92.2	—	—
jackhhao-pt, MT (262)	—	90.1	91.2	98.5	—	97.3	—	—
SPML-pt, MT (300)	—	98.7	98.3	99.3	—	98.7	—	—
HackAPrompt-pt, MT (1,000)	—	—	—	—	—	99.9	—	—
Smoke (10)	5/10	9/10	9/10	9/10	9/10	9/10	9/10	9/10

The v3 numbers are restricted to the tests that decided the criterion; deepset, jackhhao and the first 300 SPML items were measured later, in the v6 comparison (Table 10), and rikka, ShieldLM and yanis never. Five observations follow.

(a) The larger backbone moved the gold test; fine-tuning the base model barely did. The frozen-embedding probe reached 72.3% on Weni after adding native Portuguese data (the only probe round measured on Weni). Fine-tuning e5-base on a larger mix barely moved it (74.3%); fine-tuning e5-large on the *same* export and recipe reached 94.7%, a gain of 20.3 points from the change of backbone, in a single run per model. Both models were trained on the same export, with the same leakage (d), so the comparison between them is fair, although we did not test whether the larger model exploits near neighbours more. Meanwhile xTRam1, which shares a distribution with part of the training data (Section 3.1), was already at 98.9% with e5-base.

(b) The criterion was missed by one text, and the miss is not statistically meaningful. v3 detected 284 of 300 Weni items; the criterion required 285. The Wilson 95% interval (Wilson, 1927) for v3 is 91.5–96.7%, which contains 95%; at $n = 300$ the pass/fail distinction is not statistically meaningful. And (d) shows that the score itself depended on leaked training data: the near miss was not a near pass. Intervals for the other rounds on Weni are 85.7–92.6% (v5), 87.6–94.0% (mix), 69.1–78.9% (v2) and 61.2–71.8% (v4).

(c) Adding HackAPrompt with the leakage lock made both backbones worse on Weni: -7.7 points for e5-base (v4 vs v2) and -5.0 for e5-large (v5 vs v3). Our accounting of the exports suggests why the lock is the main difference between these pairs. The v4 export is the v2 export plus 4,000 HackAPrompt items (3,000 originals and 1,000 translations) minus 4,008 removed by the lock. Applied to the non-HackAPrompt data alone, the same lock removes 115 examples. If the lock behaves identically on both exports, about 3,893 of the 4,000 HackAPrompt items (97%) were removed, so v4 and v5 differ from v2 and v3 mainly by (i) the removal of 115 Weni near-duplicates and (ii) about 107 surviving HackAPrompt items. If this accounting is right,

it also supports, indirectly, the claim that Weni derives from HackAPrompt, since it would imply that almost every successful HackAPrompt attack in the export has a Weni neighbour above cosine 0.9. Both readings are inferences across two exports and await a direct per-source count (lacuna L1, Section 10). They also bear on (d): v5 lost the same 115 neighbours and still detects 89.7%, whereas v3-pub and v6 fall below 70% (Table 5); if the roughly 107 surviving HackAPrompt items are close relatives of Weni texts, v5 is leaky in another way, which the same per-source count would show.

(d) The v3 score depended on leaked training data far more than a cosine cut suggested. v3 was trained before the leakage lock (Section 3.4). Applying the lock’s rule afterwards flags 115 of its training examples; v3 detects all 58 Weni texts that have one of them as a neighbour and 226 of the other 242 (93.4%), which put the inflation at about 1.3 points. The decontaminated retraining shows that the score depended on much more than those 115 (Table 5). With the same recipe and without the test duplicates, their translations, the 115 near-duplicates and the ShieldLM rows of excluded origin, v3-pub detects 68.0% of Weni: v3 alone detects 81 texts that v3-pub misses, and the reverse happens once (McNemar $p < 10^{-6}$). On the 242 “clean” texts it detects 63.6%, a drop of 29.8 points against 13.8 on the 58 contaminated ones, and it misses most where the similarity to the 115 is lowest: every Weni text has a neighbour above cosine 0.82 among them, nine in ten above 0.85, and of the 29 texts at or below 0.85 v3-pub detects 2. The loss therefore does not run mainly through the 115. Either a cut at 0.9 on these compressed similarities leaves leaked neighbours among the other 1,643 rows removed, none of them above 0.9 to any Weni text, or those rows, duplicates of other tests and their translations that are ordinary attack examples as far as Weni is concerned, carried detection that v3-pub lost with them: it has 967 fewer attack examples than v3 (out of 9,436), 513 of them exact duplicates of other tests. We read v3’s 94.7% as inflated by leakage, but the 26.7-point drop bounds that inflation from above rather than measuring it, and a “v3 minus the 115” control, which would test the 115 directly, has not been run. The retraining also moves the in-distribution test as expected: without the xTRam1-derived rows, xTRam1 accuracy falls from 98.7% to 95.4% ($p < 10^{-6}$), and by the same amount on the 1,417 items without a duplicate in training (98.6% to 95.3%), so v3’s advantage there came from the shared distribution rather than from the duplicated items. The page failure does not change: v3-pub, like v3, warns on all 217 benign pages at every threshold (Section 5.2). Measured on the same items, a 27-point paired drop is unlikely to be run-to-run variance, although seed variance was not measured (Section 10).

(e) Calibration was not fitted for the fine-tuned models. At the hook’s deployment threshold of 0.9, v3 detects 91.0% of Weni and flags 0.13% of MASSIVE-pt and 0.7% of Dolly-pt sentences; v5 detects 86.3% of Weni at that threshold. Apart from missing the criterion by one text and from one smoke-test sentence (Section 6), every sentence-level measure made the guard look ready.

5 Page-level evaluation

5.1 Protocol

The intended deployment was a hook that runs after the agent’s web-fetch tool, truncates the fetched content at 6,000 characters, splits it into overlapping windows, and warns the agent when the maximum window probability reaches 0.9; the hook is warn-only and fail-open. The page-level evaluator calls the same code path as the hook, so it measures what the agent would actually see.

Benign pages. 217 technical documentation pages: the long description (the README rendered into package metadata) of Python packages installed in the author’s development environ-

Table 5: Leakage audit: v3, the decontaminated v3-pub and v6 on the same items. Sentence rows: accuracy in % at threshold 0.5 (for Weni, detection; for the technical sets, the share of benign texts not flagged) with Wilson 95% intervals, and exact McNemar p-values on paired predictions: “p pub/v3” compares v3-pub with v3, “p v6/pub” compares v6 with v3-pub (v6 against v3 is in Table 10). Technical sets: Stack Overflow questions and answers and Python docstrings, all benign. v3-pub is v3’s training set minus 1,758 examples (Section 3.4); v6 is v3’s minus the 115 Weni near-duplicates plus 10,000 technical benign texts (Section 7). “Clean” and “contam.” split Weni by whether a text has one of the 115 as a neighbour above cosine 0.9. Page rows: 640/160 windows on the frozen page list of Section 7.1; false alarm (FA) on the 217 benign pages, detection on the 60 injected pages, and the share of the 2,247 benign windows with $P \geq 0.5$.

Test	n	v3	v3-pub	v6	p pub/v3	p v6/pub
Weni	300	94.7 [91.5–96.7]	68.0 [62.5–73.0]	63.7 [58.1–68.9]	$< 10^{-6}$	0.18
Weni, clean	242	93.4 [89.5–95.9]	63.6 [57.4–69.4]	60.7 [54.5–66.7]	$< 10^{-6}$	0.48
Weni, contam.	58	100.0	86.2	75.9	—	0.11
xTRam1	1,500	98.7 [97.9–99.1]	95.4 [94.2–96.3]	99.3 [98.8–99.6]	$< 10^{-6}$	$< 10^{-6}$
xTRam1-pt	500	96.0 [93.9–97.4]	94.8 [92.5–96.4]	98.2 [96.6–99.1]	0.263	$< 10^{-4}$
deepset	116	94.8 [89.2–97.6]	94.0 [88.1–97.0]	88.8 [81.8–93.3]	1.000	0.070
MASSIVE-pt	1,500	99.8 [99.4–99.9]	99.9 [99.5–100]	100.0 [99.7–100]	1.000	0.500
SO questions	300	33.3 [28.2–38.8]	41.0 [35.6–46.6]	100.0 [98.7–100]	0.002	$< 10^{-6}$
SO answers	300	13.3 [9.9–17.6]	31.0 [26.0–36.4]	100.0 [98.7–100]	$< 10^{-6}$	$< 10^{-6}$
Docstrings	300	62.0 [56.4–67.3]	74.0 [68.8–78.6]	100.0 [98.7–100]	$< 10^{-5}$	$< 10^{-6}$
Page FA@0.5	217	100.0 [98.3–100]	100.0 [98.3–100]	34.6 [28.6–41.1]	—	—
Page FA@0.999	217	100.0 [98.3–100]	100.0 [98.3–100]	19.4 [14.7–25.1]	—	—
Page Det.@0.5	60	100.0 [94.0–100]	100.0 [94.0–100]	90.0 [79.9–95.3]	—	—
Windows ≥ 0.5	2,247	99.6	98.6	7.2	—	—

Table 6: Page-level results for v3.

Window (chars/overlap)	Windows (benign / injected)	False alarm on benign pages	Detection (Weni / xTRam1)
1,500 / 200	895 / 260	100.0% at every threshold	100% / 100%
640 / 160	2,258 / 662	100.0% at every threshold	100% / 100%

ments, one per package, longer than 1,500 characters, excluding any containing the word “injection”, truncated at 6,000 characters. All 217 exist on PyPI at the installed version, and 216 are byte-identical to the PEP 658 core-metadata file served by PyPI; the remaining one is rebuilt from the JSON API description with a hash check.

Pages with an injection. 60 pages: the same READMEs with one test injection inserted at a deterministic paragraph boundary, 30 drawn from Weni and 30 from the positives of the xTRam1 test split, sampled by md5. Neither set was loaded as a training source, although part of xTRam1’s training split reached training through ShieldLM and 83 of its 1,500 test items have a duplicate in v3’s training data (Section 3.1).

Decision. Page score = maximum over windows; thresholds 0.5, 0.9, 0.95, 0.99 and 0.999. We ran two window configurations: the original 1,500 characters with 200 of overlap, and 640/160, adopted after we found that the original windows exceeded the model’s 256-token context (Section 8); even so, 9.2% of the 640-character windows still exceed it (Section 5.4).

5.2 Results

All 217 benign pages raised a warning at every threshold from 0.5 to 0.999, with either window; the Wilson 95% interval for the false-alarm rate is 98.3–100%. The lowest page-level maximum among

Table 7: Same model, two units of evaluation (v3, threshold 0.9).

Unit	Benign data	False alarm
sentence	MASSIVE-pt (1,500, conversational)	0.13%
sentence	Dolly-pt (1,000, instructions)	0.7%
sentence	smoke: “run setup.exe and click next”	P = 1.0
page	217 technical READMEs	100%

benign pages was 0.9999 with 1,500-character windows and 0.9998 with 640-character windows (values stored to four decimals), and the median was 1.0. Only at a threshold of 0.9999, outside the evaluated grid, would one page stop warning (false alarm 99.54%). The 100% detection is therefore uninformative: every page fires, with or without an injection. The decontaminated v3-pub (Section 4d) fails in the same way on the frozen page list: 217 of 217 benign pages warn at every threshold, and 98.6% of its benign windows score at least 0.5 (Table 5). Leakage is not what breaks the guard in use.

The guard was rejected for the web-fetch hook, the hook stayed disabled, and the pack’s meta-data now states that it is only usable on conversational text.

5.3 What the page protocol adds

A sentence benchmark answers “does this text, alone, look like an attack?” A deployed guard answers “does any part of this page look like an attack?”, which aggregates many windows (here about 10.4 benign windows per page with 640-character windows). With a maximum over k windows, a per-window false-positive rate p becomes a page false-alarm rate of $1 - (1 - p)^k$ under independence, so even a modest per-window rate is amplified roughly k -fold. In our case the amplification is moot, because benign windows already score near 1.0; but the arithmetic explains why sentence-level false-positive rates below 1% are not reassuring for page-level use. Per-window false-positive rates were not recorded in the original run (lacuna L4); the baseline run of Section 5.4 recorded them. There, 99.6% of v3’s benign 640-character windows score at least 0.5, against 3.5–4.3% for the ProtectAI and Proventra classifiers, whose page false alarms are this amplification at work, and 0.5% for Prompt Guard 2 (86M).

5.4 Public baselines under the same protocol

We ran five public classifiers under the same page protocol: two English DeBERTa-v3 models (deepset, 2023a; ProtectAI.com, 2024), one multilingual mDeBERTa model (Proventra, 2025), and both sizes of Llama Prompt Guard 2 (Meta, 2025), the 86M model on the same mDeBERTa-v3 backbone and the 22M model on the English DeBERTa-v3-xsmall. Each reads windows sized to its own context (1,300/200 characters at 512 tokens; 640/160 at 256 tokens for ours), and a 640/160-window control was run for every public model except deepset, whose output already saturates with 1,300-character windows. Prompt Guard 2 was measured on 7 October, a day after the others, once access to its gated repository, which requires accepting the Llama 4 Community License, had been granted; its predecessor (Meta, 2024), whose access is requested separately, still returned HTTP 403 and was not evaluated. Prompt Guard 2’s configuration names its two classes only LABEL_0 and LABEL_1, so before each run we checked that LABEL_1 is the malicious class on a known injection and on a benign sentence. The Proventra model, chosen as the multilingual representative while Prompt Guard was inaccessible, stays in the comparison; its card reports injections embedded in websites and articles among its training data. All six models read the same exported pages, and

Table 8: Public classifiers under the page protocol: false alarm (FA) on the 217 benign pages and detection (Det.) on the 60 injected pages (30 Weni, 30 xTRam1), in %. “Window” is the model’s token limit and the window length in characters; the last four rows are the 640/160 control. Models: e5-large v3 (ours), ProtectAI deberta-v3-base-prompt-injection-v2, Proventra mdeberta-v3-base-prompt-injection, deepset deberta-v3-base-injection, PG2 86M and 22M = Llama-Prompt-Guard-2-86M and -22M. “Det.@0.9” is detection at 0.9, overall and (in parentheses) on the Weni / xTRam1 insertions. “AUROC” is computed over pages. “Det.@2%” is detection at the lowest threshold that leaves at most 4 of the 217 benign pages above it (FA $\leq 2\%$). “Win.” is the share of benign windows with $P \geq 0.5$ (not recorded for the control).

Model	Window	FA@0.5	FA@0.9	FA@0.99	Det.@0.9	AUROC	Det.@2%	Win.
v3 (ours)	256/640	100.0	100.0	100.0	100 (100/100)	0.857	40	99.6
ProtectAI	512/1,300	15.7	11.1	5.5	58 (97/20)	0.813	50	4.3
Proventra	512/1,300	14.7	12.4	8.3	68 (70/67)	0.883	40	3.5
deepset	512/1,300	100.0	100.0	100.0	100 (100/100)	0.829	32	100.0
PG2 86M	512/1,300	1.8	0.9	0.0	57 (90/23)	0.956	63	0.5
PG2 22M	512/1,300	0.0	0.0	0.0	7 (13/0)	0.639	23	0.0
ProtectAI	512/640	30.9	21.2	8.8	77 (97/57)	0.847	55	—
Proventra	512/640	38.2	33.6	26.7	85 (90/80)	0.845	38	—
PG2 86M	512/640	2.3	0.9	0.5	58 (93/23)	0.923	65	—
PG2 22M	512/640	0.9	0.0	0.0	18 (27/10)	0.616	22	—

Table 9: The same classifiers on sentences at threshold 0.5, and cost per full window on the 4-core ARM VM (one window per call). Our Weni figure is inflated by leakage (Section 4d); the decontaminated v3-pub detects 68.0%. Weni: detection on the 300 native pt-BR injections. xTRam1: full English test split, 2,060 sentences (650 attacks); TPR and FPR in %. $\approx$: Prompt Guard 2 was timed on x86 notebook instances (1,138 and 433 ms per window there); its ARM cost is the instance time scaled by the ratio of the Proventra model timed on the same instance, an estimate rather than a measurement.

Model	Weni det.	xTRam1 acc.	TPR	FPR	AUROC	ms/window
e5-large v3 (ours)	94.7	98.7	98.9	1.3	0.999	1,045
ProtectAI v2	99.3	94.9	84.2	0.1	0.992	868
Proventra mDeBERTa	79.0	89.6	72.6	2.6	0.906	845
deepset	100.0	48.3	98.9	75.0	0.666	879
Prompt Guard 2 86M	93.7	84.7	51.7	0.1	0.973	$\approx$ 847
Prompt Guard 2 22M	16.3	77.3	28.2	0.0	0.893	$\approx$ 305

each page is scored by the maximum over its windows of the attack log-odds, with thresholds applied to the corresponding probability; the log-odds keep a ranking where probabilities round to 1. Sentence-level numbers use the 300 Weni items and the full English xTRam1 test split (2,060 sentences, 650 attacks), with the same text hashes in every run. Our model and the first three public models ran on the same 4-core ARM cloud VM, on CPU in fp32; Prompt Guard 2 ran with the same script, library version and inputs on x86 CPU instances of a cloud notebook service, which keeps its quality numbers comparable but not its latency (Table 9). Tables 8 and 9 report the results.

The failure is not unique to our model, nor universal. The deepset classifier also warned on all 217 benign pages at every threshold up to 0.99 (Wilson 95% CI 98.3–100%), and on sentences it flags 75.0% of the benign xTRam1 items. ProtectAI v2 and the Proventra mDeBERTa model warned on 15.7% (34/217; CI 11.4–21.1%) and 14.7% (32/217; CI 10.6–20.1%) of the benign pages at 0.5, and on 5.5% and 8.3% at 0.99. Prompt Guard 2 warned on far fewer: the 86M model on 1.8% (4/217; CI 0.7–4.6%) at 0.5 and on none at 0.99, and the 22M model on none at any threshold (CI 0–1.7% at 0.5). In ProtectAI and Proventra the median benign window scores

0.00028 and 0.00123, against 0.99998 for ours, and page false alarms come from a few windows (3.5–4.3% of benign windows score at least 0.5) accumulated over about 4.9 windows per page: the amplification of Section 5.3. In Prompt Guard 2 (86M) the median benign window scores 0.004, and only 0.5% of benign windows reach 0.5. These model cards report broader training data than ours: ProtectAI lists more than twenty sources, including instructions and conversations; Proventra reports injections embedded in legitimate content such as websites and articles; and Prompt Guard 2 reports benign data from the web and an energy-based loss term on benign prompts that its card credits with better precision on out-of-distribution data.

One public model meets the page criterion, narrowly. Prompt Guard 2 (86M) is the only model whose false-alarm rate at threshold 0.5 is below 2%: 4 of 217 benign pages (1.8%; 95% CI 0.7–4.6%, so 217 pages cannot show that it stays below 2% with margin), with 62% detection (37 of 60; 49–73%). At 0.9 it warns on 0.9% and detects 57%, and its page AUROC, 0.956, is the highest in the comparison. Its detection, however, comes almost entirely from the longer Weni insertions (93%, against 30% for xTRam1), and on sentences it misses the Weni criterion (93.7% against 95%) and detects 51.7% of the English xTRam1 attacks at 0.1% false positives. Part of the xTRam1 gap is definitional: its card counts as malicious only explicit attempts to override prior instructions, and the xTRam1 attacks it misses include requests for medical records or credentials and for a poem with a prescribed opening line, without any such instruction; most of the 19 Weni sentences it misses are HackAPrompt-style obfuscations. The 22M model, whose card warns that its backbone has no multilingual pretraining, warns on no page but detects only 15% of the injected pages (Weni 20%) and 16.3% of Weni sentences. For the other models, detection at the lowest threshold that keeps false alarms at or below 2% is 23–50% under the main protocol and at most 55% in the control. Their only point in the evaluated grid below 2% false alarm with non-trivial detection is ProtectAI at threshold 0.999 (0.9% false alarm, 50% detection), and almost all of that detection also comes from the longer Weni injections (90%, against 10% for xTRam1). With 60 injected pages, the 95% interval on detection spans about ± 12 points: ProtectAI detects 38 of 60 pages at 0.5 (51–74%) and Proventra 42 of 60 (57–80%).

Sentence and page rank the models in opposite orders. On xTRam1 sentences our model has the highest accuracy (98.7%) and AUROC (0.999) of the six, on a test that is in distribution for it (Section 3.1; v3-pub scores 95.4% on the 1,500-item subset of Table 5); at page level it ties with deepset for the most false alarms. Prompt Guard 2 (86M) shows the inversion from the other side: third by xTRam1 sentence AUROC (0.973, behind ours and ProtectAI), it has the best page AUROC (0.956). The sentence benchmark did not predict behaviour in the unit in which the hook decides.

Saturation, not the threshold. 98.2% of our benign windows score at least 0.999. For deepset the output is nearly constant: page log-odds range from 6.41 to 6.79 on benign pages and from 6.65 to 6.88 on injected pages (5th–95th percentiles), so threshold 0.999 removes false alarms and detection together (0% and 0%). Our model keeps some ranking (page AUROC 0.857), but the threshold that brings false alarms to 2% sits at $P > 1 - 5 \times 10^{-6}$, which depends on the fifth decimal place of a saturated output and is not an operating point. Prompt Guard 2 (86M) is the opposite case: its benign page log-odds have median -4.9 (95th percentile -3.2) against $+3.6$ for injected pages, and its 2% operating point lies at $P > 0.18$, in the middle of the output range. The 22M model’s output is compressed on long windows (page probabilities between about 0.2 and 0.36 for almost every page), so it neither warns nor separates (page AUROC 0.639), although it separates isolated short sentences better (xTRam1 AUROC 0.893). This is consistent with a defect of training data rather than of thresholding; the v6 retraining (Section 7) tests it directly.

Window size is an operating parameter. Halving the window to 640/160 roughly doubled false alarms at 0.5 (ProtectAI 15.7% $\rightarrow$ 30.9%, Proventra 14.7% $\rightarrow$ 38.2%) and raised detection

(63% $\rightarrow$ 80% and 70% $\rightarrow$ 88%): the short xTRam1 injections (median 136 characters) are diluted less in a short window (ProtectAI page detection on xTRam1: 30% $\rightarrow$ 63%), while each page has about twice as many windows on which to err. Prompt Guard 2 (86M) barely moved (false alarm 1.8% $\rightarrow$ 2.3%, detection 62% $\rightarrow$ 65%, page AUROC 0.956 $\rightarrow$ 0.923). Even at 640 characters, 9.2% of our model’s windows still exceed its 256-token context, against 6.7% (ProtectAI, deepset, Prompt Guard 2 22M) and 1.9% (Proventra and Prompt Guard 2 86M, which share the mDeBERTa tokenizer) of the 1,300-character windows at 512 tokens.

Cost. On the ARM VM a full window takes about 1.05 s with our e5-large model (560M parameters) and 0.85–0.88 s with the DeBERTa-base models (184M); batching does not help on this CPU. An average page (4,736 characters) therefore costs about 11 s with our model (10.4 windows) and about 4 s with the DeBERTa models at 1,300 characters (4.9 windows), a material delay for a synchronous web-fetch hook. Prompt Guard 2 was timed on the notebook instances next to the Proventra model as an anchor: there the 86M model costs the same as the Proventra model (1,138 against 1,136 ms per window; same backbone) and the 22M model 36% of it, which puts them at about 4 s and 1.5 s per average page on the ARM VM by linear conversion.

Caveats. Benign pages are a single genre, and the injections are pasted text rather than hidden markup. The public models’ training data are not fully documented; if Weni derives from HackAPrompt (Section 3.4), a model trained on HackAPrompt could be inflated on Weni, and Prompt Guard 2’s card does not list its training sets either. Its narrower definition of an attack lowers its detection on both test sets and may also help keep its benign pages clean. Our model’s xTRam1 numbers carry the overlap caveat of Section 3.1. Weni is not clean for us either: our 94.7% is inflated by leakage (Section 4d), and against the decontaminated v3-pub (68.0%) the public models with the highest Weni detection, ProtectAI v2 (99.3%) and Prompt Guard 2 86M (93.7%), are far ahead, with the caveat just stated about their training data. The baseline script reproduces the 94.7% Weni detection of Section 4, which checks that it uses the same inference path as the deployed toolkit.

6 Diagnosis

6.1 Hypothesis: “imperative technical text = injection”

Our working explanation, recorded on the day of the failure, is that the model learned to treat technical, imperative text as injection because the benign training examples were conversational (MASSIVE assistant commands and Dolly instructions). Injection corpora are full of imperatives addressed to a system (“ignore”, “print”, “execute”, “reveal”); the benign corpora we used contain imperatives too, but conversational ones addressed to an assistant. Technical documentation sits on the injection side of that boundary: second-person imperatives, shell commands, configuration directives, warnings about what to override. **This explanation was a hypothesis when the failure was found.** It is consistent with the evidence below; the v6 retraining (Section 7) tests it, and supports it for technical prose while leaving residual false alarms on README markup; and the decontaminated v3-pub, which still warns on every page, rules out leakage as the cause (Section 7.2).

6.2 Evidence

The early signal was in the smoke test. The benign smoke sentence “Installation instructions: run setup.exe and click next” was classified as an injection in every round from A on where the per-item score was recorded (A, B, v3, v5), with rising confidence: $P = 0.883$ in round A, 0.967

in round B and 1.0 for v3; in v2, v4 and the mix one of the five benign smoke sentences was misclassified, but which one was not recorded. In round 0 the probe missed all five Portuguese injections (highest $P = 0.186$) and classified all five benign sentences correctly, so the sentence was not yet at issue. One technical sentence among five benign ones was the only item written to represent the deployment distribution in the whole evaluation suite, and its error was read as an isolated miss rather than a class.

Documentation was absent from training. An audit of the v6 training set, which contains 23,578 of v3’s 23,693 training texts verbatim, against the 329 package metadata texts on the machine found no training text identical to a README, no training text contained in a README (≥ 200 characters), and no README contained in the training set; 9 training texts shared a 13-gram with some README, all XML/XHTML boilerplate. The same audit of v3-pub, a strict subset of v3’s training data, against 334 metadata texts found nothing in common, not even a 13-gram. None of the benchmark READMEs, or any long fragment of them, was among the benign training examples; we did not measure how much README-like text from other sources the training data contained.

Saturation removes the threshold as a control. Because the fine-tuned models were served without temperature calibration, their outputs saturate: the smoke sentence and nearly every page receive $P = 1.0$ to four decimal places. The minimum page score of 0.9998 leaves no usable threshold: even at 0.9999 only one of the 217 benign pages would stop warning. Calibration on in-distribution validation would not by itself repair this: temperature scaling is monotonic, so it can move the threshold but not change the order in which pages are ranked; the validation split shares the conversational benign distribution; and the probe rounds, which were calibrated ($T = 1.1286$ and 1.2346), still scored the setup.exe sentence at 0.883–0.967.

The failure survives a correct window. With 1,500-character windows the classifier saw only the first 256 tokens of each window, since technical READMEs have a recorded median of about 3.04 characters per token (Section 8; the measurement script was not preserved). Shrinking the window to 640 characters, so that most windows fit the context (9.2% still exceed 256 tokens; Section 5.4), did not change the result: 100% false alarms at every threshold with both settings. Truncation was a real bug, but it is not the cause.

6.3 What the model sees

Each window enters the classifier as `query:` + up to 256 tokens of README text. A window from a typical package page contains an installation command, a usage snippet, and second-person instructions (“set this variable”, “run this command”, “do not use X in production”). From the classifier’s point of view these are short imperative strings addressed to an executing agent, which is what most training injections look like and what few training benign examples looked like. The benign side of the training data taught it what a harmless *chat* looks like, not what harmless *content* looks like. The highest-scoring benign pages in the baseline run (Section 5.4) fit this reading. For our model they are installation and test blocks, deprecation notices and tables with emoji, that is, imperative technical text; the public models fail elsewhere: ProtectAI on code that prompts the user for a secret or password and on a link to a security policy, the mDeBERTa model on code that reads environment variables and on a table of attack techniques, deepset on licence and contribution text, and Prompt Guard 2 (86M), on its four benign pages above 0.5, on fragments outside the prose: a table of check marks, badges and a header, a terminal example with an error message, and an options table. After retraining (Section 7), the windows that still fire are of another kind: badges, sponsorship links and markdown tables.

7 Fix attempt: technical benign data (v6)

7.1 Design

The v6 round tests the hypothesis of Section 6 directly by adding technical benign text to training: Stack Overflow questions and answers (koutch, 2023; pacovaldez, 2022) and Python docstrings from CodeSearchNet (Husain et al., 2019; Nan-Do, 2023), 10,000 training examples in total with 1,000 test examples per source. Package metadata is **never** used for training, because it is the page test. The v6 training set is the v3 training set minus the 115 Weni near-duplicates found by the leakage lock, plus the 10,000 technical benign examples, for a total of 33,578 (24,240 benign, 9,338 injection); HackAPrompt is not included. The positive fraction falls from 39.8% (v3) to 27.8% (v6). The recipe is otherwise identical to v3 (e5-large, 3 epochs, learning rate $2e-5$, batch 8×4 , fp16, 256 tokens, temperature 1.0 at inference).

v6 is compared with v3 on the same items: Wilson 95% intervals per test and exact McNemar tests (McNemar, 1947) on paired predictions, since both models see exactly the same texts in the same order. Evaluation repeats v3’s: the same tests, texts, code and ARM machine. The tests v3 had not been run on (deepset, jackhhao, SPML and the three technical sets) were run for both models on the same first 300 items. Pages are the frozen list used in Section 5.4, with the same fingerprint for both models: between the two runs, new packages in the author’s environments had changed 21 of the 217 pages selected live and altered 2 others, which would have invalidated the comparison. Probabilities were stored to four decimals, which limits threshold analysis near 0 and 1. Acceptance requires a page-level false-alarm rate below 2% with detection reported.

A decontaminated control. v6 changed three things at once: it added technical negatives, dropped the 115 Weni near-duplicates and lowered the positive fraction. v3-pub (Section 3.4), trained a day later, is a partial control. Like v6 it lacks the 115 Weni near-duplicates, and unlike v6 it has no technical negatives; but it also drops the test duplicates, their translations and the ShieldLM rows of excluded origin that v6 keeps (1,758 examples removed against 115), and its positive fraction is 38.6% against v6’s 27.8%. Comparing v6 with v3-pub therefore isolates the technical negatives better than comparing v6 with v3, but it is not an ablation. v3-pub was evaluated on the same items, frozen pages, code and machine as v3 and v6, and paired against both.

7.2 Results

Table 10 compares v3, v3-pub and v6 on sentences and Table 11 on pages; Table 5 gives the intervals and paired tests for the main rows.

The diagnosis is supported for technical prose. On held-out technical benign sentences, v3 flagged 66.7% of Stack Overflow questions, 86.7% of answers and 38.0% of docstrings; v6 flags none of the 900. Decontamination alone helps much less: v3-pub, without technical negatives, still flags 59.0%, 69.0% and 26.0% of them (51.3% pooled, against 63.8% for v3). These tests come from the same sources as v6’s new training data, so they are in distribution for v6; the page test is not, since no README entered training (Section 6.2). On the READMEs, the share of benign 640-character windows scoring at least 0.5 fell from 99.6% (v3) and 98.6% (v3-pub) to 7.2% (v6), a fourteen-fold reduction.

The fix is not enough for the hook. Page false alarms fell from 100% to 34.6% at threshold 0.5 and to 19.4% at 0.999, and detection on the injected pages became informative (90.0% and 81.7%). Because v3-pub, decontaminated but without technical negatives, still warns on every benign page, this drop is attributable to the technical negatives and not to the removal of leaked examples. The page criterion remains out of reach: 26 of the 217 benign pages (12.0%) still score

Table 10: v3, v3-pub and v6 on sentences at threshold 0.5: accuracy in % (for Weni, detection), on the same items for the three models, with the exact McNemar test of v6 against v3 on paired predictions (the paired tests of v3-pub are in Table 5 and in the text). v3’s Weni figures are inflated by leakage (Section 4d). Some n differ from Table 4: the deepset, jackhhao, SPML, Dolly and technical tests use their first 300 items (all items when fewer). “Weni, clean” excludes the 58 texts with a near-duplicate in v3’s training data. The technical tests are held out by hash from v6’s new training sources.

Test	n	v3	v3-pub	v6	v6 – v3 (p.p.)	McNemar p, v6/v3
Weni, pt-BR native	300	94.7	68.0	63.7	−31.0	$< 10^{-6}$
Weni, clean	242	93.4	63.6	60.7	−32.6	$< 10^{-6}$
xTRam1, en	1,500	98.7	95.4	99.3	+0.7	0.031
xTRam1-pt, MT	500	96.0	94.8	98.2	+2.2	0.007
MASSIVE-pt, benign	1,500	99.8	99.9	100.0	+0.2	0.250
Dolly, benign en	300	99.7	99.7	99.3	−0.3	1.000
Dolly-pt, benign MT	1,000	99.3	99.8	99.6	+0.3	0.250
deepset, en	116	94.8	94.0	88.8	−6.0	0.016
deepset-pt, MT	116	93.1	92.2	89.7	−3.4	0.125
jackhhao, en	262	98.9	99.2	99.2	+0.4	1.000
jackhhao-pt, MT	262	98.1	98.9	98.5	+0.4	1.000
SPML, en	300	100.0	100.0	100.0	0.0	1.000
SPML-pt, MT	300	100.0	100.0	100.0	0.0	1.000
SO questions, benign technical	300	33.3	41.0	100.0	+66.7	$< 10^{-6}$
SO answers, benign technical	300	13.3	31.0	100.0	+86.7	$< 10^{-6}$
Docstrings, benign technical	300	62.0	74.0	100.0	+38.0	$< 10^{-6}$
Smoke (correct/10)	10	9/10	9/10	10/10	—	1.000

Table 11: v3 against v6 on pages (640/160 windows, frozen page list): false alarm (FA) on the 217 benign pages and detection (Det.) on the 60 injected pages, in %, with Wilson 95% intervals for v6. v3 warns on every benign page and detects every injected page at every threshold (intervals 98.3–100% and 94.0–100%), and so does v3-pub (Table 5).

Threshold	FA v3	FA v6 [95% CI]	Det. v3	Det. v6 [95% CI]	Det. v6, Weni / xTRam1
0.5	100.0	34.6 [28.6–41.1]	100.0	90.0 [79.9–95.3]	90.0 / 90.0
0.9	100.0	30.4 [24.7–36.8]	100.0	85.0 [73.9–91.9]	86.7 / 83.3
0.95	100.0	29.0 [23.4–35.4]	100.0	85.0 [73.9–91.9]	86.7 / 83.3
0.99	100.0	24.0 [18.8–30.1]	100.0	83.3 [72.0–90.7]	86.7 / 80.0
0.999	100.0	19.4 [14.7–25.1]	100.0	81.7 [70.1–89.4]	83.3 / 80.0

1.0000 at the stored resolution, so no threshold brings false alarms to 2%. Page AUROC is 0.868 for v6; computed the same way on four-decimal probabilities, v3 gets 0.505 (v3-pub 0.500) because almost all of their pages tie at 1.0, while its log-odds in the baseline run give 0.857 (Table 8). The quality of the ranking is therefore similar; what changed is that the ranking now lies in a usable range of the output. Against the public baselines, v6 warns on more benign pages at 0.5 than ProtectAI v2 and the mDeBERTa model (34.6% against 15.7% and 14.7%) and detects more injected pages (90% against 63% and 70%): it sits elsewhere on the same trade-off rather than beyond it. At the same 640/160 windows, Prompt Guard 2 (86M) ranks pages better than v6 (AUROC 0.923 against 0.868, the latter computed on four-decimal probabilities) and operates at 2.3% false alarm with 65% detection at threshold 0.5. The windows that still fire, a median of 2 per flagged page, are README boilerplate absent from Stack Overflow and docstrings: HTML badges, sponsorship and donation links, markdown tables and link lists.

The Weni drop is not clearly the fix’s. On Weni, v6 detects 63.7% against v3’s 94.7% (94 texts against 1; McNemar $p < 10^{-6}$), and 60.7% against 93.4% on the 242 clean texts. We first read this as the price of removing the shortcut. The decontaminated control undercuts that reading: v3-pub, which has no technical negatives, detects 68.0% (63.6% on the clean texts). Against it, v6 loses 4.3 points, with 47 texts detected only by v3-pub and 34 only by v6 (McNemar $p = 0.18$; on the clean texts 39 against 32, $p = 0.48$). One reading is that what v6 and v3-pub share, the loss of the 115 training examples closest to Weni, accounts for most of both drops, and the technical negatives for little at threshold 0.5. The totals fit it; the pattern of errors fits it less well. v5 lost the same 115 neighbours and kept 89.7%, although it also gained about 107 HackAPrompt attacks that may be Weni’s relatives (Section 4c). Neither model loses most where the similarity to the 115 is highest (above 0.9, v3-pub detects 50 of 58 texts and v6 44), and the two part ways where it is lowest: of the 29 Weni texts least similar to the 115, v6 detects 20 and v3-pub 2. And the two models miss partly different texts: 62 are missed by both, 47 by v6 alone and 34 by v3-pub alone. v3-pub’s loss may therefore run through its broader cuts (Section 4d), which v6 does not share, in which case the two drops have different causes and v6’s owes more to its new negatives than the 4.3 points suggest. The bases also differ (Section 7.1). The “v3 minus the 115” control would decide between these readings. In ranking, the cost of the new negatives is clearer. Paired with assistant-style negatives, v3-pub still separates Weni well (AUC 0.977; 95.7% detection at a 2% false-positive rate), so its errors at 0.5 are mostly calibration; v6 separates it worse (AUC 0.883; 77.0%) and gives $P = 0.0000$ to 23% of Weni texts, against 4.3% for v3-pub, so that lowering the threshold barely recovers them (67.0% at 0.1 and 69.3% at 0.01, against 73.0% and 78.0% for v3-pub). Paired with the technical negatives the order flips: v3 and v3-pub detect none of Weni’s injections at a 2% false-positive rate (AUC 0.843 and 0.655, at the stored resolution), while v6 detects 77% (AUC 0.885). The texts v6 loses relative to v3 are short (705 characters on average, against 1,101 for the rest) and framed as legitimate tasks (“generate a movie title about the sentence...”, “translate...”, “you are a search engine...”), with the attack’s target phrase split or obfuscated. The other tests with attacks hold or improve (xTRam1-pt 96.0% $\rightarrow$ 98.2%, $p = 0.007$; jackhhao and SPML unchanged), except English deepset (94.8% $\rightarrow$ 88.8%, $p = 0.016$), where v3-pub stays at 94.0% (v6 against v3-pub: $p = 0.07$).

Reading. Of the two effects that our first reading conflated, one can now be told apart firmly; the other remains open. Firmly: the technical negatives, not decontamination, are what removes the false alarms on technical text and pages; they take technical false alarms from v3-pub’s 26–69% to 0% and page false alarms from 100% to 34.6% at threshold 0.5, while decontamination alone lowers technical false alarms only partly (from 63.8% to 51.3% pooled) and changes nothing on pages. Open: what causes most of v6’s Weni drop. A decontaminated model without technical negatives drops almost as far at 0.5, so the comparison with v3 does not show that the negatives caused it; the pattern of errors and v5 argue against a shared cause, and in ranking against conversational

Table 12: Integration defects.

#	Symptom	Root cause	Fix
I1	The hook fed the model mojibake (“mÃ³dulo”)	On Windows, Python decodes stdin as cp1252; the agent sends UTF-8. Mojibake looks like an injection to the classifier	read <code>sys.stdin.buffer</code> , decode UTF-8 explicitly
I2	Hook output with accents broke	<code>json.dumps(..., ensure_ascii=False)</code> on a cp1252 stdout	ASCII JSON with <code>\u</code> escapes
I3	The hook was silent when served by the remote server	The remote server ran an older version that answered binary decisions in another format; the hook’s key lookup raised an exception that the fail-open handler swallowed	read the probability field present in both formats; redeploy the server
I4	Binary classifier answered as a multi-choice decision	single response format in the classifier client	binary labels answer with the binary decision type
I5	Window larger than the model	1,500 characters exceed 256 tokens at about 3.04 characters per token: silent truncation	640/160 windows; boundary test; 9.2% of windows still exceed 256 tokens

negatives the negatives’ cost is visible. The control that isolates the 115 Weni neighbours has not been run. None of the three versions meets both criteria: v3 misses Weni by one text with an inflated score and fails the page criterion, and v3-pub and v6 fail both. v3 remains in the toolkit for conversational text only, and the hook stays disabled.

8 Engineering lessons for agent integration

The guard was tested end to end before being enabled, and that test found three defects in the hook (I1–I3) before the page-level failure was even visible; a related response-format defect in the classifier client (I4) had been fixed earlier the same day. We list the defects that generalise to any classifier placed inside an agent loop (Table 12; the full incident log is in Appendix B).

Lesson 1: encoding errors are adversarial-looking inputs. I1 is the most instructive. A guard is trained to fire on odd, out-of-place text; mojibake is odd, out-of-place text. An encoding bug in the glue code therefore does not just degrade accuracy, it manufactures false positives that look like the guard doing its job.

Lesson 2: fail-open needs a heartbeat. Fail-open is the right default for a warn-only guard: a broken guard must not break the agent. But I3 shows the cost: a version mismatch between client and server turned every call into an exception, every exception into “no warning”, and the hook into a silent no-op that looked healthy. A fail-open component must report that it failed, for example with a counter or a periodic self-test on a known positive, otherwise “no warnings” is indistinguishable from “all clear”.

Lesson 3: pin and test the contract, not just the model. I3 and I4 are contract bugs between a client and a decision server across versions. A contract test that sends one known injection and one known benign text through the real deployment path, against each server version in use, would have caught both. The same applies to the training environment: a library update in the cloud notebook image removed a training argument and broke the first fine-tuning run, and a dataset library update removed loading scripts.

Lesson 4: measure the window in tokens. I5 is an instance of a general trap: the

integration layer measures text in characters, the model in tokens, and the ratio depends on the genre. Technical READMEs measured a median of 3.04 characters per token (10th percentile 2.61), so a 1,500-character window is about 493 tokens and roughly half of each window was discarded. The measurement script was not preserved (lacuna L5). The 640-character windows chosen to fit still left 9.2% of windows over the limit in the baseline run (median 196 tokens, maximum 646), because the ratio varies within a page as well as across genres. Windows should be cut in tokens; the released evaluator still cuts them in characters, as in this study, but counts the tokens of every window and reports the share that exceeds the model’s context (Section 13).

Lesson 5: evaluate through the deployment code path. The page evaluator calls the hook’s own scoring function. Every defect above would have been invisible to an evaluator that called the model directly.

Lesson 6: freeze the evaluation inputs and keep raw scores. The benign pages were first selected live from the installed packages. By the time v6 was evaluated, new packages had changed 21 of the 217 pages and altered 2 more, and only a frozen, fingerprinted export kept the v3–v6 comparison valid. Scores were stored as probabilities rounded to four decimals, which hid v3’s residual page ranking (visible only in log-odds, Section 5.4) and leaves v6’s 2% operating point undefined (Section 7.2). Store hashes of the inputs and the raw logits.

After the fixes, the hook has 9 test cases, and the hook, classifier and embedding-cache suites (13 tests) pass.

9 Discussion

9.1 What benchmark accuracy measured here

The numbers in Table 4 were produced with fixed partitions, a criterion committed before measurement, a leakage lock from v4 on, and no tuning on test sets, with the overlaps of Section 3.1 as known exceptions. They were nonetheless inflated: the best Weni score depended on training data that a decontamination pass removes (Section 4d). And they answered a different question from the one that mattered. The benchmarks measured whether the model separates injections from mostly *conversational* benign text, because that is what the benign side of the tests was largely made of: MASSIVE-pt and Dolly-pt are short requests to an assistant (Dolly enters through its instruction field only), the English sets pair injections mostly with chat-like prompts, and Weni contains no benign items at all. The only technical benign item written into the suite was one smoke-test sentence, and from round A on it was wrong in every round where its score was recorded (Section 6.2). The public baselines make the same point from outside our pipeline: on xTRam1 sentences our model ranks first of six, helped by in-distribution data, and at page level it ties for last (Section 5.4).

Five concrete corrections follow for anyone evaluating an injection classifier for agent use:

1. **The benign side of the test must be drawn from the deployment distribution.** For a web-fetch or file-reading guard that means documentation, code, configuration, logs and ordinary web pages, not chat. The one public model in our comparison that keeps page false alarms near 2%, Prompt Guard 2 (86M), is also the one whose card explicitly reports benign web data in training (Section 5.4).
2. **Evaluate in the unit of decision.** If the deployed decision is “warn on this page”, the metric is the page false-alarm rate at the deployed threshold, with detection measured on the same pages with an inserted attack.
3. **Report intervals and margins.** At $n = 300$ the 95% criterion could not be distinguished

from the result (Section 4b); and the 20-point capacity effect between e5-base and e5-large on the same (leaky) data dwarfs most of the differences between data recipes we tried.

4. **Decontaminate against every test, and check the audit by retraining.** A cosine cut at 0.9 against the gold test, applied after the fact, put the leakage in our best model at 1.3 points; retraining without the duplicates of every test, their translations and the gold test’s near-duplicates cost 26.7, and more on the texts the cut had called clean (29.8 points) than on the others (13.8; Section 4d). An audit against the gold test alone cannot see what duplicates of the other tests contribute, and embedding similarities are compressed, so a threshold that looks strict can separate little. Check every test, translated copies included: exact duplicates after normalisation turned up in nearly every test we used (Section 3.1).
5. **Separate effects before attributing them.** We first credited v6’s Weni drop to the new negatives; a decontaminated control without them dropped almost as far, so the credit was premature, although that control does not settle the question either (Section 7.2). A fix aimed at false alarms must be checked against the gold set again, and against a control that differs from it in the fix alone.

9.2 Typed decision models with agents: a dialogue with Akita (2026)

Akita’s post (Akita, 2026a) gives practical criteria for when a typed decision model is the right tool. We agree with them, and our guard satisfies them on paper: a single repeated judgment (every fetched page), a closed action list (warn or not), surrounding code applying exact rules (the hook), and errors that can be routed to review (the agent, and ultimately the user, decides). The guard also followed the advice, given in the essay inside the post, not to give such a model sole authority over destructive actions: it was warn-only and fail-open by design. That a design meeting all these criteria still failed completely is, we think, the useful addition. Specifically:

Recalibrating on one’s own examples is necessary but not sufficient. The essay in the post reports a hosted model stating high confidence when it has no information, and advises recalibrating with one’s own labelled examples. Our fine-tuned models had the analogous defect (uncalibrated, saturated outputs). But recalibrating on *our own validation split* would not have helped, because that split came from the same conversational distribution. The examples one calibrates on must be drawn from the inputs the model will see in use, and for a content guard those are not the examples one naturally has.

Training a classifier for the specific job requires defining the job by its input distribution. The earlier post names a classifier trained for the task as one alternative to a hosted decision API (Akita, 2026b). We did exactly that, and the classifier was excellent at the job as defined by its training labels. The job as defined by its inputs (arbitrary fetched content, mostly technical and benign) was never represented. For decision models in general, the question to ask is not only “what are the options?” but also “what does the negative class look like in use?”

Confidence-based escalation collapses under saturation. A common integration pattern routes high-confidence decisions to automation and low-confidence ones to a stronger model or a human. With probabilities pinned at 1.0, the escalation band is empty: every page is a confident, wrong, “easy” case. v6 shows the mirror image: 23% of Weni’s attacks receive $P = 0.0000$, confident and wrong in the other direction (4.3% for v3-pub). Confidence-based routing is only as good as the calibration *under the deployment distribution*.

Fixing the negative class may move the boundary for the positive class, but check before concluding that it did. We first read v6’s 31-point loss on Weni as the price of the repair: attacks phrased as ordinary tasks becoming indistinguishable from technical instructions.

The decontaminated control (Section 7.2) loses almost as much without any new negatives: at threshold 0.5 v6 is 4.3 points below it, within noise, while in ranking against conversational negatives the gap is larger (AUC 0.977 against 0.883); how much of the loss the negatives cause is still open. When two classes share their surface form, more data on one side can shift the boundary through the other, and a typed interface does not tell the integrator when that is happening; but a drop on a gold set is evidence of a moved boundary only after leakage has been ruled out.

Silent failure is a property of the integration, not the model. The evidence in the post concerns the model’s behaviour; our most dangerous bug (I3) had nothing to do with the model. A guard that fails open and silently looks exactly like a guard that finds nothing.

9.3 Relation to concurrent work

Liu (2026) re-evaluates fifteen public detectors on agent benchmarks (AgentDojo, τ -bench, BIPIA) and on benign text that includes about 2,500 GitHub README chunks of 80 to 220 words. It reports that deepset’s classifier flags every README chunk and is unusable, that removing HTML markup from the READMEs cuts the false-positive rate of two other detectors by more than half, and that false positives on ordinary text do not predict those on tool outputs. Its recommendations are to measure false positives on the agent’s own inputs, report detection at a fixed low false-positive rate, check whether the detector was trained on the benchmark, and prefer detectors trained on inputs like the ones they will screen. It appeared on 2 October, while our study was under way, and we found it only during the literature review; the two were carried out independently.

We read our study as an independent replication of that gap, with three differences. First, a different language and a detector we trained ourselves. Its training data are known, so the gap could be traced to the composition of the benign side and the diagnosis tested by retraining (Section 7), which a study of third-party models cannot do. Where the two studies overlap, they agree: deepset warns on every page in our protocol as on every chunk in Liu’s. Our leakage result is also a concrete case of Liu’s third recommendation, to check whether a detector was trained on the benchmark: our own model, with a leakage audit in the pipeline, had been trained on duplicates and near neighbours of its tests (Section 4d). Second, the unit of evaluation: we score whole pages through the code path of a deployed hook, which exposes the amplification over windows (Section 5.3), the window-size trade-off (Section 5.4) and the integration defects of Section 8. Third, the framing: our classifier was deployed as a typed decision, and Section 9.2 draws the consequences for that family of models.

Liu’s markup result bears directly on ours. The windows v6 still flags are badges, sponsorship links and markdown tables (Section 7.2), and the four benign pages on which Prompt Guard 2 (86M) warns are also non-prose fragments (a check-mark table, badges, a terminal transcript, an options table; Section 6.3). Both observations are consistent with markup and other non-prose content, rather than imperative prose, being the main remaining source of false alarms once technical text is covered. We did not measure our models with and without markup. Doing so, and adding README boilerplate from a source disjoint from the benchmark as negatives, is the next experiment.

9.4 On negative results

The guard stays out of the web-fetch hook; v3 remains in the local toolkit for conversational text only, and the toolkit continues with other decisions. We report the failure in detail because the path that led to it (open data, careful splits, a pre-fixed criterion, a strong benchmark result) is the path most practitioners would follow, and the step that caught the problem, evaluating in the unit of use

on ordinary content, is cheap and still rarely reported; a recent exception measures detectors on benign English README chunks rather than whole pages (Liu, 2026). The attempted fix and the decontaminated retraining are reported for the same reason: the first worked on what it targeted, and the second showed that a drop of nearly the same size happens without the fix once the leaked data leave the training set, so the price we had attributed to the fix is not established.

10 Limitations

- **Single run per configuration.** No seed variance was measured; differences of a few points between rounds (e.g., v3 vs v5, or v3-pub vs v6 on Weni) may be noise. The 26.7-point paired difference between v3 and v3-pub is large for a seed effect, but this was not measured.
- **Leakage in the best model.** v3 was trained before the leakage lock; 115 of its training examples are near-duplicates of Weni texts. A post-hoc cut at cosine 0.9 put their effect at 1.3 points; the decontaminated retraining loses 26.7, more on texts without such a neighbour (29.8 points) than on those with one (13.8; Section 4d). Which of v3-pub’s removals carries the effect, and how much of it is leakage rather than lost attack data, is not isolated: it also removed 967 attack examples, 513 of them exact duplicates of other tests, and the “v3 minus the 115” control has not been run. The per-source breakdown of the 4,008 examples removed in v4 was not recorded, and the accounting in Section 4(c) is an inference across exports (lacuna L1).
- **xTRam1 is not out of distribution** for v2–v6, and exact duplicates of test items were in v3’s training data for nearly every test, among them 20 of the 262 jackhhao items and 83 of the 1,500 xTRam1 items (Section 3.1). Only v3-pub is free of them; on xTRam1 it scores 95.4% against v3’s 98.7%.
- **Incomplete evaluation of v3.** v3 was measured on deepset, jackhhao and the first 300 SPML items only in the v6 comparison, and never on rikka, ShieldLM or yanis; numbers for some of these that appear in the deployed pack’s metadata have no traceable origin and are not used here.
- **Intervals.** Wilson intervals are reported for Weni, the page-level rates, the pages of the v3–v6 comparison and the rows of Table 5; the other tests of Tables 4 and 10 lack intervals in the text (lacuna L6; v3pub.json has them).
- **Language.** The Portuguese benign set (MASSIVE) is European Portuguese; machine translations were not reviewed by a human.
- **Weni provenance.** That Weni is translated HackAPrompt material is supported only indirectly (Sections 3.4 and 4c). A recorded error analysis of v2 on Weni exists only in a commit message, without script or counts (lacuna L7).
- **Page benchmark scope.** Benign pages are a single genre (Python package READMEs) selected from the packages installed in one environment; injected pages reuse the same READMEs, so detection is uninformative while every README fires. The injections are pasted text, not hidden markup, and with 60 injected pages the detection intervals span about ± 12 points. The original run recorded neither per-window rates (the baseline run did) nor which server instance answered (lacuna L4).
- **Unverified measurements.** The characters-per-token figure (lacuna L5), the translation speed-up (commit message only), the effective use of both GPUs and the effective batch size

(lacuna L10) are not independently verified.

- **Data revisions.** Datasets were loaded without pinned revisions; the revisions recorded in the artifacts were captured after the runs.
- **Data card warnings.** The SPML card advises against training detectors on it; we used it and flag this as a threat to validity.
- **Causal claims only partly tested.** v6 changed three things at once (technical negatives, removal of the 115 Weni near-duplicates, class balance) in a single run. The decontaminated v3-pub is a partial control, but the bases are not identical: v6 keeps the test duplicates, their translations and the ShieldLM rows of excluded origin that v3-pub removes, and v3-pub removes 1,758 examples where v6 removes 115 (Section 7.1). The cause of v6’s Weni drop therefore remains open until the “v3 minus the 115” control is run. v6’s technical test sets are in distribution for it, and we did not measure the effect of markup (Sections 7.2 and 9.3).
- **Score resolution.** Our evaluation stored probabilities rounded to four decimals, which limits threshold and AUROC analysis near 0 and 1 (Section 7.2); only the baseline run stored log-odds.
- **Baselines.** Prompt Guard v1 was not evaluated (gated access). The public models’ training data are not fully documented and may overlap Weni (through HackAPrompt) or xTRam1. Prompt Guard 2 flags only explicit attempts to override instructions, a narrower notion of attack than the test sets’, so its detection measures that difference as well as errors. Latencies come from a single 4-core ARM VM, except Prompt Guard 2’s, measured on x86 notebook instances and converted by a same-instance anchor whose ratio varied from 1.18 to 1.82 with window and batch, so they indicate only the order of magnitude.
- **Truncation.** All our models read at most 256 tokens per window, and 9.2% of the 640-character windows exceed that (the public baselines in Section 5.4 read 512).

11 Conclusion

A Brazilian Portuguese prompt-injection classifier trained only on open data passed its sentence benchmarks, missed its pre-fixed gold-set criterion by one text, and raised a false alarm on every technical documentation page it was shown. Both halves of that result turned out to matter. The benchmark number was inflated: retrained without test duplicates and near-duplicates, the same recipe detects 68.0% of the native pt-BR attacks instead of 94.7%. A cosine-similarity audit had put the inflation at 1.3 points; the 26.7-point drop bounds it from above, since the retrained model also has fewer attack examples. The use failure is real and separate: the decontaminated model still warns on every page. It is not peculiar to our model, since a public classifier does the same, and the unit of evaluation changes the verdict: sentence and page benchmarks ranked the six models in opposite orders. Only one public model, Llama Prompt Guard 2 (86M), stays near 2% page false alarms (1.8%, with 62% detection); it is also the one whose card reports benign web data in training, and it still detects only 30% of the short injections and misses our sentence criterion on Weni, so it narrows the problem rather than closing it. Adding technical benign text removed the false alarms on technical sentences and most of those on pages; against the decontaminated control it cost 4.3 points of Weni detection at threshold 0.5, within noise, so the drop we had attributed to it cannot be charged to it on the comparison with v3 alone; a control that isolates the Weni neighbours would tell how much of it is the fix’s. No version we trained meets both criteria. The practical conclusions are modest: decontaminate against every test and check the audit by

retraining; draw the benign side of the test from the deployment distribution; evaluate in the unit of decision through the deployment code path; freeze the inputs and keep raw scores; and separate effects before attributing them. The hook stays disabled.

12 Ethics and data

All training and evaluation data are public. Licenses were read from the Hugging Face Hub API, dataset cards and LICENSE/NOTICE files on 6 October 2026. Two evaluation sets (Weni and xTRam1) declare no license; we load them only for evaluation, download them at run time, and do not redistribute them or their translations. However, 577 rows of xTRam1’s training split reached our training data indirectly, through ShieldLM (Section 3.1); the v3-pub weights, trained on 7 October, exclude them, together with the ShieldLM rows whose licence differs between origin and aggregator (Section 3.4). Dolly and Stack Overflow content are CC-BY-SA; the v3-pub weights, trained with Dolly and without Stack Overflow, are published under MIT following the precedent of models trained on the same data, and we declare this position explicitly. Llama Prompt Guard 2 was evaluated under the Llama 4 Community License as a baseline only: its outputs were not used to train, calibrate or distil our models, and its weights are not redistributed (the copy used to run it sits in private storage of the notebook service).

The page benchmark does not redistribute any README: it ships package names, versions and hashes, and a builder downloads the pages from PyPI and verifies them. No personal data, private text or organisational data was used in this study. The released model detects injections and is not an attack tool; we do not release attack text beyond what the cited public datasets already contain.

A guard with a 100% false-alarm rate is harmful in a subtle way: users learn to ignore its warnings. The model card opens with the page-level failure and reports v3-pub’s own numbers; v3’s leakage-inflated 94.7% appears there only as a comparison.

Use of AI assistance. Much of the experimental code, the logging, and the first draft of this paper were produced with the help of an LLM coding agent. Every number reported is traced to a log line, result file or commit; claims that could not be traced are listed as limitations.

13 Artifacts

Code, results and the page benchmark are released at <https://github.com/alucardigo/passing-the-benchmark>; the fine-tuned weights are published separately, on Kaggle Models.

- **Code** (Apache-2.0): training and export pipeline, deterministic translation with cache, fixed-weight ensemble, page benchmark builder and evaluator, a reference agent hook (disabled by default), tests. Repository: <https://github.com/alucardigo/passing-the-benchmark>. The public repository starts without the development history.
- **Weights** (MIT): v3-pub (Section 3.4, Table 5) and the round-B probe, published on Kaggle Models as passing-the-benchmark-guard. The card opens with the page-level failure and reports v3-pub’s own numbers (Weni 68.0%, xTRam1 95.4%, 217 of 217 page false alarms). v6 does not meet the page criterion (Section 7.2) and is not released as a guard.
- **Results** (CC-BY-4.0): one JSON per round with the SHA-256 of the weights and page-level probabilities by index, the per-item probabilities of the v3–v6 and v3-pub comparisons, and the per-page and per-sentence log-odds of the baseline run, without third-party text.

- **Page benchmark:** a manifest of 217 benign pages and 60 insertions (package, version, SHA-256 of metadata and page; source, revision, md5 and position of each insertion) plus a builder that refuses mismatched hashes, following the Decision Index pattern. The evaluator accepts any Hugging Face classifier, cuts windows in characters, counts their tokens and reports the share that exceeds the model’s context, and reports false alarms and detection per threshold with Wilson intervals.
- **Attribution file** listing every dataset, model and revision with its license.

References

- Fabio Akita. Entendendo o hype do Jev: pra que serve? AkitaOnRails blog, 2026-10-03, October 2026a. URL <https://akitaonrails.com/2026/10/03/entendendo-o-hype-do-jev-pra-que-serve/>. In Portuguese; includes an essay by Paulo Câmara written for the post (he discloses that he co-founded a company competing in the same market).
- Fabio Akita. Por que coisas como TypeSafe IA não me interessam. AkitaOnRails blog, 2026-09-16, September 2026b. URL <https://akitaonrails.com/2026/09/16/por-que-coisas-como-typesafe-ia-nao-me-interessam/>. In Portuguese.
- Diogo Almeida. Introducing system one models & Jev. TypeSafe AI blog, 2026-09-15, September 2026. URL <https://typesafe.ai/blog/introducing-system-one-models-and-jev>. The FAQ states that the “System One” name draws on Kahneman’s System 1 / System 2 distinction.
- apolinario. Decision index: reproduction kit for the typed-decision benchmark suite. GitHub repository <https://github.com/apolinario/decision-index>; leaderboard data in the Hugging Face Space `multimodalart/jev-decision-index` (file `data/index-v0.2.1.json`, generated 2026-09-28), 2026. URL <https://github.com/apolinario/decision-index>. Version 0.2.1 (2026-09-27); MIT (code); chance-corrected skill score, unanswered requests count as wrong.
- Michelle Chen, Alex Reneau, and Kevin Flansburg. Introducing Clef: our open-source decision models, and new RL fine-tuning platform. Cloudflare blog, 2026-10-01, October 2026. URL <https://blog.cloudflare.com/clef-decision-models/>. Clef (frozen Qwen3.8-27B) and Clef-flash (frozen Qwen3.5-9B) with LoRA adapters and a scoring head; weights on Hugging Face under Apache-2.0.
- Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending against prompt injection with structured queries. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 2383–2400. USENIX Association, 2025. ISBN 978-1-939133-52-6. URL <https://www.usenix.org/conference/usenixsecurity25/presentation/chen-sizhe>. Preprint first posted February 2024.
- Cloudflare. Clef-flash. Hugging Face model `Cloudflare/clef-flash`, 2026. URL <https://huggingface.co/Cloudflare/clef-flash>. Apache-2.0; based on Qwen/Qwen3.5-9B; revision 17f0b0ad.
- Mike Conover, Matt Hayes, Ankit Mathur, Jianwei Xie, Jun Wan, Sam Shah, Ali Ghodsi, Patrick Wendell, Matei Zaharia, and Reynold Xin. Free Dolly: Introducing the world’s first truly open instruction-tuned LLM. Databricks blog, 2023-04-12, April 2023. URL <https://www.databricks.com/blog/2023/04/12/dolly-first-open-commercially-viable-instruction-tuned-llm>. Dataset `databricks/databricks-dolly-15k`, CC-BY-SA-3.0, revision bdd27f4d.
- Convai Innovations. Laya: a non-autoregressive typed-decision engine. GitHub repository <https://github.com/NandhaKishorM/laya>; Hugging Face models `convaiinnovations/laya` and `convaiinnovations/laya-multilingual`, 2026. URL <https://github.com/NandhaKishorM/laya>. Apache-2.0; English checkpoint on ModernBERT-large (421M), multilingual checkpoint on mmBERT-base (322M); PyPI package `laya`.
- Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian

- Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*, pages 82895–82920, 2024. doi: 10.52202/079017-2636. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html.
- Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design, 2025. URL <https://arxiv.org/abs/2503.18813>.
- deepset. deberta-v3-base-injection. Hugging Face model `deepset/deberta-v3-base-injection`, 2023a. URL <https://huggingface.co/deepset/deberta-v3-base-injection>. MIT; fine-tuned from microsoft/deberta-v3-base on deepset/prompt-injections; revision 80dda00d.
- deepset. prompt-injections. Hugging Face dataset `deepset/prompt-injections`, 2023b. URL <https://huggingface.co/datasets/deepset/prompt-injections>. Apache-2.0; 546 train / 116 test; English and German; revision 4f61ecb0.
- dmilush. shieldlm-prompt-injection. Hugging Face dataset `dmilush/shieldlm-prompt-injection`, 2026. URL <https://huggingface.co/datasets/dmilush/shieldlm-prompt-injection>. MIT (curation; sources keep their own licenses); revision dd660ae7.
- Jack FitzGerald, Christopher Hench, Charith Peris, Scott Mackie, Kay Rottmann, Ana Sanchez, Aaron Nash, Liam Urbach, Vishesh Kakarala, Richa Singh, Swetha Ranganath, Laurie Crist, Misha Britan, Wouter Leeuwis, Gokhan Tur, and Prem Natarajan. MASSIVE: A 1M-example multilingual natural language understanding dataset with 51 typologically-diverse languages. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4277–4302. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.acl-long.235. Preprint posted April 2022; data: <https://huggingface.co/datasets/AmazonScience/massive> (CC-BY-4.0), used via the mirror `mteb/amazon_massive_intent` revision 940fd47a.
- Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020. doi: 10.1038/s42256-020-00257-z.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec ’23)*, pages 79–90. ACM, 2023. doi: 10.1145/3605764.3623985.
- Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. Annotation artifacts in natural language inference data. In Marilyn Walker, Heng Ji, and Amanda Stent, editors, *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 107–112, New Orleans, Louisiana, June 2018. Association for Computational Linguistics. doi: 10.18653/v1/N18-2017. URL <https://aclanthology.org/N18-2017/>.
- Helsinki-NLP. opus-mt-tc-big-en-pt. Hugging Face model `Helsinki-NLP/opus-mt-tc-big-en-pt`, 2022. URL <https://huggingface.co/Helsinki-NLP/opus-mt-tc-big-en-pt>. CC-BY-4.0; transformer-big, target tokens >>por<< and >>pob<<; revision 9f2863d8.
- Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. Defending against indirect prompt injection attacks with spotlighting, 2024. URL <https://arxiv.org/abs/2403.14720>.
- Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. CodeSearchNet challenge: Evaluating the state of semantic code search, 2019. URL <https://arxiv.org/abs/1909.09436>.
- imoxto. prompt_injection_hackaprompt_gpt35. Hugging Face dataset `imoxto/prompt_injection_hackaprompt_gpt35`, 2023. URL https://huggingface.co/datasets/imoxto/prompt_injection_hackaprompt_gpt35. Mirror of HackAPrompt (gpt-3.5-turbo); no license field in the card metadata; revision efc8f438.

- intfloat. multilingual-e5-large. Hugging Face model `intfloat/multilingual-e5-large` (and `multilingual-e5-base`), 2023. URL <https://huggingface.co/intfloat/multilingual-e5-large>. MIT; revisions 3d7cfbda (large) and d1287505 (base); 559,890,946 and 278,044,162 parameters per safetensors metadata.
- jackhhao. jailbreak-classification. Hugging Face dataset `jackhhao/jailbreak-classification`, 2023. URL <https://huggingface.co/datasets/jackhhao/jailbreak-classification>. Apache-2.0; revision 2f2ceeb3.
- Dennis Jacob, Hend Alzahrani, Zhanhao Hu, Basel Alomair, and David Wagner. PromptShield: Deployable detection for prompt injection attacks. In *Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy (CODASPY '25)*, pages 341–352. ACM, 2025. doi: 10.1145/3714393.3726501.
- Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, New York, 2011. ISBN 978-0-374-27563-1.
- Pang Wei Koh, Shiori Sagawa, Henrik Marklund, Sang Michael Xie, Marvin Zhang, Akshay Balsubramani, Weihua Hu, Michihiro Yasunaga, Richard Lanus Phillips, Irena Gao, Tony Lee, Etienne David, Ian Stavness, Wei Guo, Berton Earnshaw, Imran Haque, Sara M. Beery, Jure Leskovec, Anshul Kundaje, Emma Pierson, Sergey Levine, Chelsea Finn, and Percy Liang. WILDS: A benchmark of in-the-wild distribution shifts. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 5637–5664. PMLR, 2021. URL <https://proceedings.mlr.press/v139/koh21a.html>.
- koutch. stackoverflow_python. Hugging Face dataset `koutch/stackoverflow_python`, 2023. URL https://huggingface.co/datasets/koutch/stackoverflow_python. Stack Overflow content, CC-BY-SA; no license field in the card metadata; revision ee6f3955.
- Hao Li and Xiaogeng Liu. InjecGuard: Benchmarking and mitigating over-defense in prompt injection guardrail models, 2024. URL <https://arxiv.org/abs/2410.22770>. Preprint of PIGuard (li2025piguard); introduces the NotInject benchmark (339 benign sentences with trigger words).
- Hao Li, Xiaogeng Liu, Ning Zhang, and Chaowei Xiao. PIGuard: Prompt injection guardrail via mitigating overdefense for free. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 30420–30437, Vienna, Austria, July 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.acl-long.1468. URL <https://aclanthology.org/2025.acl-long.1468/>.
- Yupei Liu, Yuqi Jia, Runkeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 1831–1847. USENIX Association, August 2024. ISBN 978-1-939133-44-1. URL <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei>.
- Zhuowen Liu. Passing the test you trained on: Re-evaluating prompt-injection detectors for LLM agents, 2026. URL <https://arxiv.org/abs/2610.03448>. Submitted 2026-10-02; code: <https://github.com/lzwhehe/benign-instruction-bench>.
- R. Thomas McCoy, Ellie Pavlick, and Tal Linzen. Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference. In Anna Korhonen, David Traum, and Lluís Màrquez, editors, *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3428–3448, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1334. URL <https://aclanthology.org/P19-1334/>.
- Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.
- Meta. Prompt guard 86m. Hugging Face model `meta-llama/Prompt-Guard-86M`, 2024. URL <https://huggingface.co/meta-llama/Prompt-Guard-86M>. Llama 3.1 license; first released July 2024.
- Meta. Llama prompt guard 2. Hugging Face models `meta-llama/Llama-Prompt-Guard-2-86M` and `-22M`; model card in <https://github.com/meta-llama/PurpleLlama>, 2025. URL <https://huggingface.co/meta-llama/Llama-Prompt-Guard-2-86M>. Llama 4 Community License;

- 86M on mDeBERTa-base, 22M on DeBERTa-xsmall; 512-token window.
- Yanis Miraoui. `prompt_injections`. Hugging Face dataset `yanismiraoui/prompt_injections`, 2023. URL https://huggingface.co/datasets/yanismiraoui/prompt_injections. Apache-2.0; 7 languages incl. Portuguese; revision `bd55359f`.
- Nan-Do. `code-search-net-python`. Hugging Face dataset `Nan-Do/code-search-net-python`, 2023. URL <https://huggingface.co/datasets/Nan-Do/code-search-net-python>. Python subset of CodeSearchNet (`husain2019codesearchnet`); revision `39db9186`.
- pacovaldez. `stackoverflow-questions`. Hugging Face dataset `pacovaldez/stackoverflow-questions`, 2022. URL <https://huggingface.co/datasets/pacovaldez/stackoverflow-questions>. Stack Overflow content, CC-BY-SA (card metadata says Apache-2.0); revision `869802e5`.
- Jared Palmer. `kev-9b`. Hugging Face model `jaredpalmer/kev-9b`, 2026. URL <https://huggingface.co/jaredpalmer/kev-9b>. Apache-2.0; LoRA adapter and head over Qwen/Qwen3.5-9B-Base; revision `db029f08`.
- Fábio Pérez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models, 2022. URL <https://arxiv.org/abs/2211.09527>. ML Safety Workshop, NeurIPS 2022.
- ProtectAI.com. Fine-tuned DeBERTa-v3-base for prompt injection detection. Hugging Face model `protectai/deberta-v3-base-prompt-injection-v2`, 2024. URL <https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2>. Apache-2.0; English only; revision `90c9989b`; citation as given on the model card.
- Proventra. `mdeberta-v3-base-prompt-injection`. Hugging Face model `proventra/mdeberta-v3-base-prompt-injection`, 2025. URL <https://huggingface.co/proventra/mdeberta-v3-base-prompt-injection>. MIT; fine-tuned from `microsoft/mdeberta-v3-base`; revision `b8a89d30` on 2026-10-06.
- Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. Do ImageNet classifiers generalize to ImageNet? In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 5389–5400. PMLR, 2019. URL <https://proceedings.mlr.press/v97/recht19a.html>.
- Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of NLP models with CheckList. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault, editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4902–4912, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.442. URL <https://aclanthology.org/2020.acl-main.442/>.
- rikka-snow. `prompt-injection-multilingual`. Hugging Face dataset `rikka-snow/prompt-injection-multilingual`, 2024. URL <https://huggingface.co/datasets/rikka-snow/prompt-injection-multilingual>. MIT; 11 languages incl. Portuguese; revision `flad1f3d`.
- Sander Schulhoff, Jeremy Pinto, Ansum Khan, Louis-François Bouchard, Chenglei Si, Svetlana Anati, Valen Tagliabue, Anson Kost, Christopher Carnahan, and Jordan Boyd-Graber. Ignore this title and HackAPrompt: Exposing systemic vulnerabilities of LLMs through a global prompt hacking competition. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4945–4977, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.302. URL <https://aclanthology.org/2023.emnlp-main.302/>.
- Reshabh K. Sharma. `SPML_Chatbot_Prompt_Injection`. Hugging Face dataset `reshabhs/SPML_Chatbot_Prompt_Injection`, 2024. URL https://huggingface.co/datasets/reshabhs/SPML_Chatbot_Prompt_Injection. MIT; revision `02ce8084`; accompanies the SPML paper (`sharma2024spml`).
- Reshabh K. Sharma, Vinayak Gupta, and Dan Grossman. SPML: A DSL for defending language models against prompt attacks, 2024. URL <https://arxiv.org/abs/2402.11755>.
- Yusuf Khalid Shire and Sang-Chul Kim. PIDS-Bench: Evaluating prompt-injection detectors under

- over-defense, obfuscation, and distribution shift. *IEEE Access*, 14:134184–134205, 2026. doi: 10.1109/ACCESS.2026.3728186.
- Jörg Tiedemann. The Tatoeba translation challenge – realistic data sets for low resource and multilingual MT. In *Proceedings of the Fifth Conference on Machine Translation*, pages 1174–1182, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.wmt-1.139. URL <https://aclanthology.org/2020.wmt-1.139/>.
- Jörg Tiedemann and Santhosh Thottingal. OPUS-MT – building open translation services for the world. In *Proceedings of the 22nd Annual Conference of the European Association for Machine Translation*, pages 479–480, Lisboa, Portugal, November 2020. European Association for Machine Translation. URL <https://aclanthology.org/2020.eamt-1.61/>.
- Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Multilingual E5 text embeddings: A technical report, 2024. URL <https://arxiv.org/abs/2402.05672>.
- Weni. prompt-injections-1.0.0. Hugging Face dataset Weni/prompt-injections-1.0.0, 2024. URL <https://huggingface.co/datasets/Weni/prompt-injections-1.0.0>. No license declared; 300 Portuguese attack prompts; revision 9707fa47.
- Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.
- xTRam1. safe-guard-prompt-injection. Hugging Face dataset xTRam1/safe-guard-prompt-injection, 2024. URL <https://huggingface.co/datasets/xTRam1/safe-guard-prompt-injection>. No license declared; synthetic, generated with GPT-3.5-turbo; 8,240 train / 2,060 test; revision a3a877d6.
- Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. Benchmarking and defending against indirect prompt injection attacks on large language models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, V.1*, pages 1809–1820. ACM, 2025. doi: 10.1145/3690624.3709179. Preprint first posted December 2023 (BIPIA).
- Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-acl.624.

A Compute and cost

Resource	Measurement
Cloud notebook, 2× T4, fine-tuning sessions	v2 1,995.9 s (includes a failed e5-large attempt) · v3 8,511.4 s · v4 1,848.3 s · v5 8,120.5 s · total 20,476 s ≈ 5.69 h
Cloud notebook, v6 fine-tuning	12,713 s ≈ 3.53 h (the first identical attempt ended in an error without a log)
Cloud notebook, v3-pub fine-tuning	7,056 s ≈ 1.96 h
Free GPU quota of the notebook service	about 30 h per week
CPU embeddings for probes	2,901 s (round 0) + 2,777 s (A) + 4,917 s (B) = 10,595 s ≈ 2.94 h
ARM cloud VM (4 OCPU, aarch64), e5-large inference	about 19.6 texts per minute
ARM cloud VM, page protocol (idle, 4 threads)	per full window: 1,045 ms (e5-large, 256 tokens), 845–879 ms (DeBERTa-base, 512 tokens); about 11 s and 4 s per average page
Weights (fp16, zipped)	e5-base 515,813,767 B; e5-large 1,031,696,719 B

B Incident log

#	Incident	Root cause	Fix
I6	First fine-tuning run failed	notebook image’s library update removed <code>warmup_ratio</code>	<code>warmup_steps</code> = 6% of steps
I7	e5-large out of memory on T4	trained after e5-base in the same session; fragmented memory with batch 16×2	e5-large alone, batch 8×4 , gradient checkpointing, expandable segments
I8	Translation about 7 s per short instruction	greedy decoding looping to a fixed 512-token cap	cap at $1.6 \times$ input length
I9	Lost translations	killed process lost the buffered batch	flush per batch
I10	Local evaluation crashed (OpenBLAS, exit 139)	memory pressure on the workstation	single-threaded BLAS in the server; fallback to the ARM VM
I11	ARM translation of e-mails took over an hour for 16 texts	long generation on ARM CPU	character and example caps; no generation on ARM
I12	Each round recomputed every embedding	no cache	SHA-1 $\rightarrow$ vector cache, public data only
I13	Dataset with a loading script would not open	<code>datasets</code> 4 removed loading scripts	download the data file directly
I14	Failed attacks would become benign	HackAPrompt holds attacks only; label implicit	keep successful attacks only
I15	Remote server declared dead	2 s health timeout over a slow VPN round trip	10 s timeout for remote servers
I16	Mixed-length batches paid about $3 \times$ in padding	no length sorting	sort by length in the classifier
I17	Install script broken on the Linux VM	CRLF line endings exported by git	<code>.gitattributes</code> with LF for scripts

C Timeline

Date (2026, UTC−3)	Event
1 Oct	First probe: xTRam1 86.3%, 0/5 Portuguese smoke injections detected
2 Oct	Local translation pipeline; final criterion committed (Weni $\geq 95\%$, xTRam1-pt $\geq 90\%$)
5 Oct	Round A (xTRam1-pt 81.8%); round B (Weni 72.3%); v2 (Weni 74.3%); leakage lock; hook ready but disabled
6 Oct	v4 (Weni 66.7%); v5 (89.7%); v3 re-evaluated (94.67%, later shown to depend on leaked training data); mix (91.3%); v3 chosen, status experimental
6 Oct	Hook defects found; page-level evaluation: 100% false alarms at both window sizes; guard rejected for web fetch
6 Oct	v6 export with technical benign data started
6 Oct	Public baselines under the page protocol; v6 trained and evaluated against v3 on frozen pages; hook kept disabled
7 Oct	Llama Prompt Guard 2 (86M, 22M) under the page protocol; v3-pub trained and evaluated against v3 and v6 (Weni 68.0%; 100% page false alarms)